

The Loading Dock, Explained

The Dock's eight objectives, one part each, rewritten for a reader with very little background — story first, plain-word glossary second, code third, the exam questions last. All eight parts in one volume.

THE EIGHT PARTS

- 1 The Standard Connector**
Objective 3.7 · MCP, direct API or agent-to-agent — count the clients and the churn
- 2 The Bloat Audit**
Objective 3.1 · Every tool schema rides in every prompt — trim by logs AND consequence
- 3 The Gate Pass and the Manifest**
Objective 3.2 · authN is who you are; authZ is which bays — the wildcard is the ghost
- 4 Shelving the SOPs — Chunking and Indexing**
Objective 3.5 · Chunk along meaning; label every box inside; two indexes; contextual retrieval
- 5 Three Questions, Three Routes — Retrieval by Shape**
Objective 3.6 · Hybrid search; ID → lookup, count → aggregate, prose → RAG; RAG cannot count
- 6 The Dials — Accuracy and Latency**
Objective 3.3 · Underline the fixed currency; justify with constraint + dial + direction + number
- 7 The Dock Cameras — Observability**
Objective 3.4 · Logs = WHAT, traces = WHERE, metrics = NORMAL?; the mean lied; ids never label metrics
- 8 The Catalogue at the Gate — Progressive Discovery**
Objective 3.8 · Map first, territory on demand — except small + stable + always-used + cached

COMBINED EDITION · HOW TO USE THIS VOLUME

Contents

Each part is self-contained and keeps its own page numbers (Part N · p. X in the footer). Read them in this order — it is the Dock's chapter order, and each part leans on the one before it. Every part has the same shape: the story first, the words a layman needs, the concept drawn, the code line by line as one continuous tale, the law, the traps, that objective's Dock Book questions explained, and a 'say it in one breath' card.

Part	Title	Objective	Starts at
Part 1	The Standard Connector MCP, direct API or agent-to-agent — count the clients and the churn	Objective 3.7	volume p. 4
Part 2	The Bloat Audit Every tool schema rides in every prompt — trim by logs AND consequence	Objective 3.1	volume p. 13
Part 3	The Gate Pass and the Manifest authN is who you are; authZ is which bays — the wildcard is the ghost	Objective 3.2	volume p. 20
Part 4	Shelving the SOPs — Chunking and Indexing Chunk along meaning; label every box inside; two indexes; contextual retrieval	Objective 3.5	volume p. 28
Part 5	Three Questions, Three Routes — Retrieval by Shape Hybrid search; ID → lookup, count → aggregate, prose → RAG; RAG cannot count	Objective 3.6	volume p. 38
Part 6	The Dials — Accuracy and Latency Underline the fixed currency; justify with constraint + dial + direction + number	Objective 3.3	volume p. 48
Part 7	The Dock Cameras — Observability Logs = WHAT, traces = WHERE, metrics = NORMAL?; the mean lied; ids never label metrics	Objective 3.4	volume p. 55
Part 8	The Catalogue at the Gate — Progressive Discovery Map first, territory on demand — except small + stable + always-used + cached	Objective 3.8	volume p. 64

The five laws of Domain 3 — and where each one lives

#	The law	In one breath	Part
1	Every tool you add is a door; know who holds the key.	Token cost (schema × every call), confusion cost (lookalikes), security surface. Trim by logs AND consequence.	2 + 3
2	Chunk along meaning, not length.	Cut on headings; write the origin INTO the chunk; size ≈ query granularity; overlap is a priced dial.	4
3	Retrieval follows the shape of the data, not the fashion.	ID → lookup · count → aggregate · procedure prose → hybrid · codes → keyword-lean · symptoms → semantic-lean. RAG cannot count.	5
4	You cannot fix what you cannot see.	Logs = WHAT, traces = WHERE, metrics = NORMAL?; SLAs are promises about p95s, not means; ids never label metrics.	7
5	Send the map first, the territory on demand.	Catalogue + fetch — EXCEPT small + stable + always-used + cacheable → monolith + caching wins.	8

The numbers to keep (gold on the Memory Map)

Number	What it is	Part
2,314 vs 811 tokens/call · 60M tokens/day	9-tool belt vs 3-tool belt; the belt tax at 40,000 calls	2
35% → 49% → 67% (5.7 → 3.7 → 2.9 → 1.9%)	Retrieval failure-rate cuts: contextual embeddings, + contextual BM25, + reranking	4
retrieve ~150 → rerank → top-20 · +~200 ms	The reranking shape; 20 beat 5 and 10	4, 5, 6
≤ 200k tokens (~500 pages)	Skip-RAG threshold: whole corpus + prompt caching	4, 8
mean 310 ms / p95 2,900 ms	The mean lied; SLAs live in the tail	7
₹1,800 vs ₹4 per error · ₹6 ceiling	Tier by consequence; justify with both numbers	6
362,460 vs 4,625 vs 4,500	Monolithic vs progressive vs small + cached, tokens per call	8
trigger ~100k · keep 3 · 1,024–4,096 min cache	Context editing defaults; caching minimums by tier (Haiku 4.5 = 4,096)	8
Spec 2026-07-28	MCP: stateless, stdio + Streamable HTTP, sampling/logging deprecated, elicitation current	1

PART 1 OF 8 · OBJECTIVE 3.7

CARRIED OVER FROM CCAR-F

The Standard Connector

MCP, a plain API call, or talking agent-to-agent — how Claude gets plugged into the TMS, the WMS and the carriers, and how to choose which plug to use

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- What an API is, what MCP is, and why MCP is like the shipping container
- The three things an MCP server offers — tools, resources, prompts — and which one to use for what
- How to read the little Kaveri MCP server line by line, in plain words
- The one sentence that decides MCP vs direct call vs agent-to-agent: count the clients and the churn
- What changed in the MCP spec in July 2026, and the five exam questions on this objective

PART 1 · OBJECTIVE 3.7 · CARRIED OVER FROM CCAR-F, NOW GRADED AT PROFESSIONAL WEIGHT

The standard connector — MCP, and when not to use it

The board approved a magnificent crane on Friday. Monday morning it stands on the dock, gleaming — connected to nothing. A crane that lifts nothing is a sculpture. This part is about the cables: how the AI gets plugged into the software that actually knows where the trucks are.

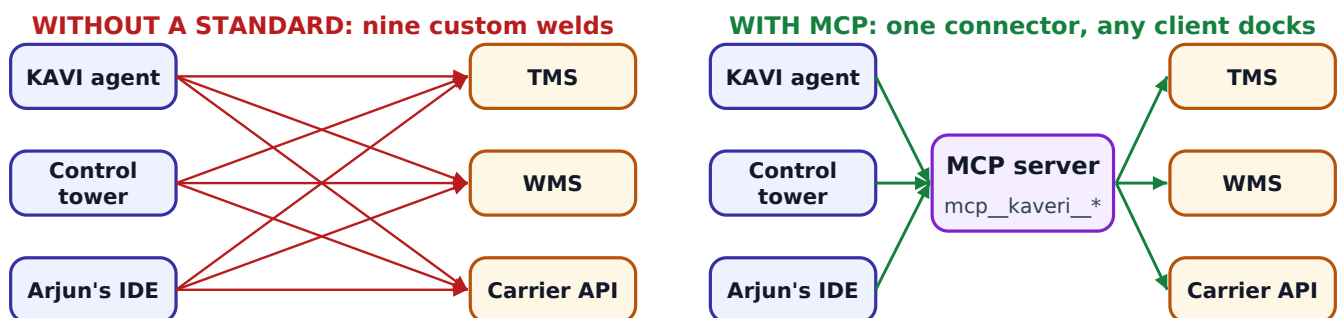
↑ ONE DOCK CONNECTOR, NOT FIVE WELDED FORKLIFTS

Before shipping containers were standardised (1956), every port needed custom cranes for every ship, and every ship needed custom holds for every cargo. The container did not make cargo better — it made **every pairing cheap**: any ship, any crane, any truck, any train. **MCP is the container standard for tools and data**: one protocol, and any AI client can dock with any server. But standards have a price. If you own one ship and one crane and will never own another, welding them together directly is less machinery than a container system. Objective 3.7 grades whether you know *which* situation you are in.

First, the words a layman needs

Three pieces of software live at Kaveri. The **TMS** (transport management system) knows where every truck and consignment is. The **WMS** (warehouse management system) knows what is on which shelf. The **carrier APIs** are the windows of BlueDart, Delhivery and IndiaPost through which you ask 'where is my parcel?'. The AI (KAVI, the Kaveri assistant) is useless unless it can ask those three systems questions and get answers back.

An **API** is simply a counter window between two programs. You slide a form through in an agreed format ('consignment id: KF-2481'), and a filled form comes back ('status: CUSTOMS_HOLD'). Every software system has its own window with its own form. That is the problem: five systems, five different forms, and every new AI client would have to learn all five.



add a 4th client → 3 more welds; change a tool → fix 3 clients

add a 4th client → it docks; change a tool → clients discover it

Nine custom welds versus one connector. The saving grows with every client and every change.

Word	In plain words	The dock picture	The exam-grade line
API	A counter window where one program asks another for something, in a fixed format.	The carrier's enquiry counter: slide in a consignment id, get a status back.	An interface with a contract (inputs, outputs). A direct API call = code calling that window straight, no protocol layer.

Word	In plain words	The dock picture	The exam-grade line
MCP	Model Context Protocol — one standard way for an AI to discover and use tools, documents and templates.	The shipping container: one shape, every crane fits.	Open protocol; server exposes tools / resources / prompts; clients discover with */list and call. Naming: mcp_server_tool.
MCP server / client	The server is the program offering the tools; the client is the AI-side program using them.	Server = the dock connector; client = the truck that plugs in.	One server, N clients. Clients discover capabilities; additions propagate without client code changes.
Tool	An action the AI can ask the server to perform (or live data to fetch).	A forklift — it moves something.	Executable primitive; schema from type hints, description from docstring; the docstring is what the model reads to decide.
Resource	A piece of data (a document) the AI can read into its context. No action happens.	The manifest on the rack — you read it, nothing moves.	Data-not-action primitive; addressed by URI (sop://214); read via resources/read — no tool call spent.
Prompt (MCP sense)	A reusable, server-kept template for how to do a job.	The laminated procedure card on the wall.	Interaction template fetched by the client; versioned once on the server instead of pasted into forty heads.
Transport	The road the messages travel on: a local pipe, or the network.	Same-dock chute (stdio) vs the highway (Streamable HTTP).	stdio = local process, OS identity. Streamable HTTP = remote, standard HTTP auth (OAuth recommended). SSE transport is gone.
Discovery (*/*list)	The client asks 'what do you offer?' and gets a menu back.	Reading the dock's noticeboard before asking for anything.	tools/list, resources/list, prompts/list; then tools/call or resources/read. Keeps clients current when the list changes.
Agent-to-agent	Two AIs that each have judgement talk to each other with goals and proposals, not function calls.	Ringling the other company's dispatcher, not pressing a button on their machine.	Use when the far side negotiates/decides. Send a structured brief with authority limits; budget the exchange; verify replies.
Elicitation	The server pauses and asks the human user a question mid-task.	The gate guard phoning the office: 'this one has no paperwork — proceed?'	Current client primitive (spec 2026-07-28). Sampling and logging are deprecated.

The three kinds of furniture an MCP server offers

Every MCP server can offer three things, and the exam loves asking which one fits a need. Think of the dock:

TOOLS <i>the forklifts</i> Do something / fetch live data. get_shipment_status("KF-2481") → {status: CUSTOMS_HOLD}	RESOURCES <i>the manifest rack</i> Documents to READ into context. sop://214 → the full SOP text. No action, no side effects.	PROMPTS <i>the laminated cards</i> Reusable procedure templates. customs_triage("KF-2481") → "check status, read sop://214..."
---	--	---

Tools ACT · Resources INFORM · Prompts TEMPLATE. Three primitives, one sentence.

Simple example. A dispatcher asks KAVI: 'What does SOP 214 say about e-way bills?' The lazy design makes a **tool** called read_sop — that spends a tool call (a round-trip, tokens, time) just to read a file. The right design offers SOP 214 as a **resource** (sop://214) that the client reads straight into context. Rule of thumb: **if nothing happens in the world and you only want to read**

something, it is a resource. If you want to *do* something (check live status, book a slot), it is a tool. If you want to hand the model a *procedure to follow*, it is a prompt.

The code, line by line — the little Kaveri server grows up

Below is the whole server. It is short on purpose. Hand-type it; read the story column as one continuous tale of a dock getting its first standard connector.

```
e1_kaveri_mcp.py
1  from fastmcp import FastMCP
2  import json, pathlib
3
4  mcp = FastMCP("kaveri")          # server name -> mcp__kaveri__*
5
6  @mcp.tool()
7  def get_shipment_status(consignment_id: str) -> dict:
8      """Live status for one Kaveri consignment id."""
9      return {"id": consignment_id, "status": "CUSTOMS_HOLD",
10             "missing": "e-way bill"}
11
12  @mcp.tool()
13  def find_alternate_carriers(lane: str) -> list:
14      """Carriers serving a lane, best first."""
15      return ["BlueDart", "Delhivery"]
16
17  @mcp.resource("sop://214")        # data, not action
18  def sop_214() -> str:
19      """SOP 214 - customs holds and e-way bill re-filing."""
20      return pathlib.Path("sops/sop_214.md").read_text(encoding="utf-8")
21
22  @mcp.prompt()
23  def customs_triage(consignment_id: str) -> str:
24      """Laminated card: how to work a customs-hold ticket."""
25      return (f"Work customs hold for {consignment_id}: check status, "
26             "read sop://214, list options, never promise a date.")
27
28  if __name__ == "__main__":      # the import-probe guard
29      mcp.run()                   # stdio transport by default
```

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-2	The story begins: Arjun opens the toolbox. fastmcp is the library that knows the MCP standard so he does not have to; pathlib is for reading files. <i>Example:</i> like buying a ready-made container instead of welding one — the standard's shape comes in the box.
4	The dock gets a name. One server object, called kaveri. That name becomes the namespace: every tool on this server will show up to any client as mcp__kaveri__get_shipment_status. <i>Example:</i> the painted sign over the gate — every forklift inside gets a 'KAVERI-' number plate.
6-10	The first forklift. The decorator @mcp.tool() turns an ordinary Python function into a discoverable tool. Where does the tool's contract come from? The name from the function name, the schema (what inputs it takes) from the type hint consignment_id: str, and the description from the docstring. <i>The docstring IS the interface</i> — it is the sentence the model reads when deciding whether to call this tool. A lazy docstring here is a mis-dialled phone call later. <i>Example:</i> KAVI reads 'Live status for one Kaveri consignment id', sees the question 'where is KF-2481?', and picks this forklift. It returns a small dictionary: status CUSTOMS_HOLD, missing e-way bill.
12-15	The second forklift. Same pattern: give it a lane like 'Pune→Nagpur', it returns carriers best-first. Two tools now — the demo version had these two too; nothing new yet.

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
17-20	The manifest rack. <code>@mcp.resource("sop://214")</code> puts a document on the rack under an address. A client that wants SOP 214 <i>reads</i> it — no action, no side effects, and crucially no tool-call round-trip. <i>Example:</i> the dispatcher does not send a forklift to fetch the rulebook; the rulebook is on the rack, she reaches over and reads it. Mixing tools and resources up is a classic 3.7 distractor shape.
22-26	The laminated card. <code>@mcp.prompt()</code> registers a template. Ask for <code>customs_triage("KF-2481")</code> and you get back the procedure sentence: check status, read <code>sop://214</code> , list options, never promise a date. Written once, versioned on the server, instead of pasted into forty dispatchers' heads. <i>Example:</i> when the procedure changes, you reprint one card, not forty.
28-29	The guard. If someone <i>imports</i> this file (e.g. a test, or the free probe <code>python -c "import el_kaveri_mcp"</code>), <code>__name__</code> is not <code>"__main__"</code> , so <code>mcp.run()</code> never fires and nothing starts — the probe stays free and silent. Run it as a script and the server speaks stdio : a local pipe, one client, no network. The remote flavour is Streamable HTTP — Part 3 puts a gate pass on it.

WHAT PRINTS

```
python -c "import el_kaveri_mcp"          # silence: the guard held
python el_kaveri_mcp.py                  # server runs on stdio, waits
# from a client: tools/list -> 2 tools · resources/list -> sop://214
# prompts/list -> customs_triage
```

🔧 TRY IT YOURSELF · PREDICT BEFORE RUN

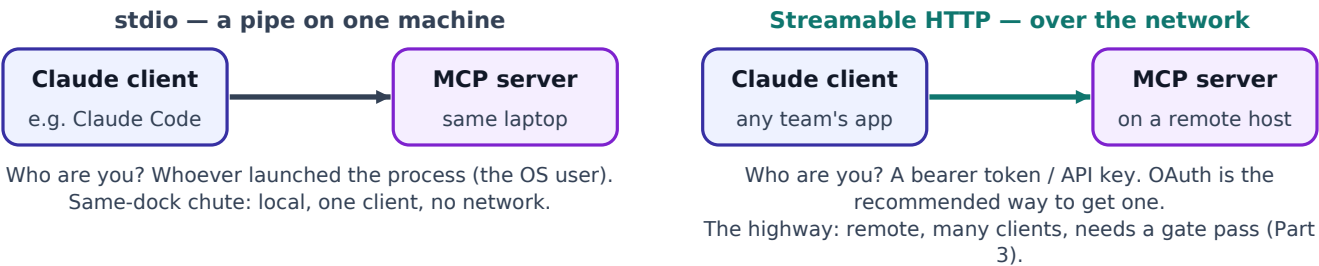
Before running anything, write two predictions in your notebook: (1) What prints when you run the import probe, and why does the server NOT start? (2) A client wants to answer 'what does SOP 214 say?' *without* spending a tool call — which of the three primitives does it use?

Answers to check against: (1) Nothing prints — the `__main__` guard held. (2) The **resource** `sop://214`, read with `resources/read`.

⚡ SABOTAGE LAB · BREAK EXACTLY ONE THING

Change line 7's type hint to `consignment_id: int`. Predict: does the server crash, or does the generated schema silently change — and what will the model now try to send? Probe with `tools/list`, compare the `inputSchema` before and after, then fix it back. **Lesson:** in FastMCP the type hints write the contract. A wrong hint is a wrong phone book — nothing crashes, the model just starts dialling numbers instead of ids.

Two roads: local pipe or the network



SSE as a transport is GONE from the current spec (2026-07-28). Only stdio and Streamable HTTP remain.
Transport shapes authentication: the pipe inherits the OS user; the highway needs a token.

⇌ **DRIFT · THE DOCS MOVED**

What older teaching says: The client and server perform an *initialize* handshake and stay stateful; *sampling* and *logging* are core client primitives; remote servers use the *SSE* transport.

What the current docs say (checked Sep 2026): Spec 2026-07-28: the protocol is **stateless** — every request carries version and capabilities in `_meta`, with an optional `server/discover` request replacing the old handshake. **Sampling and logging are deprecated** (integrate the LLM directly; log to `stderr` or `OpenTelemetry`). Change notifications are opt-in via `subscriptions/listen`. Transports are **stdio and Streamable HTTP** — *SSE-as-transport* is gone. The client primitive still current: **elicitation**. If a question quotes the old teaching, the correct option states both readings and dates the current one.

The decision — MCP, direct API, or agent-to-agent?

Three carriers, three different right answers. This is Atlas A4 as a picture:

MCP SERVER
BlueDart — rich API, four internal clients need it

- one connector, N clients
- discovery keeps clients current
- tool list can change safely
- remote: Streamable HTTP + OAuth

Hurts: overhead never repaid when exactly one client will ever call

DIRECT API / CLI
IndiaPost — one endpoint, one caller, tight budget

- least machinery
- no protocol layer to operate
- latency: one hop
- wrap as MCP LATER if clients multiply

Hurts: becomes N custom welds the day a second client appears

AGENT-TO-AGENT
Delhivery's booking agent — negotiates slots, has judgement

- exchange goals, not function calls
- structured handoff brief
- authority limits travel in the brief
- budget the conversation

Hurts: absurd for a lookup — a phone call to ask the time

Same company, three integrations, three mechanisms — each correct for its own reasons.

How do you decide? **Count two things: the clients and the churn.** How many different programs will need this integration? And how often does the list of tools change?

MANY CLIENTS →

ONE CLIENT →

TOOL LIST CHANGES

MANY clients · list CHANGES
MCP server
the container pays off: N clients × changing list

ONE client · list CHANGES
Direct call now; wrap as MCP later
one weld; re-weld only if a 2nd client arrives

TOOL LIST STABLE

MANY clients · list STABLE
MCP server (still)
N clients still dock once; discovery is a bonus

ONE client · list STABLE
Direct API call
a standard buys nothing here — the weld is fine

...and if the far side has JUDGEMENT (it negotiates, proposes, refuses) → agent-to-agent, whatever the cell

The 3.7 sentence as a grid. The judgement test sits outside the grid.

The question says...	Choose	Because
Many clients (KAVI, control tower, Arjun's IDE) must reach the same tools; the tool list changes	MCP server	One connector, N clients; discovery keeps clients current
One fixed integration, one owner, tight latency budget, no reuse foreseen	Direct API/CLI call in code	A standard buys nothing here; a weld is less machinery

The question says...	Choose	Because
The other side is itself an agent with judgement (a carrier's booking agent negotiating slots)	Agent-to-agent	You are exchanging goals and proposals, not calling functions
A carrier offers only a REST API and you control the agent side	Wrap the API as an MCP tool	The weld hides behind the standard connector once, then N clients benefit

A worked example in plain words. Kaveri has four internal programs that all need 'get shipment status', and Arjun adds a new tool roughly every month. Without MCP, every new tool means editing four programs (4 welds × 12 months = 48 edits a year). With one MCP server, he adds the tool once and all four clients discover it on their next `tools/list`. Now the opposite case: IndiaPost has one endpoint, exactly one caller (the ETA job), and the list never changes. Standing up an MCP server there is a container crane for a rowing boat — build it later, only if a second caller ever shows up.

⚠️ **TRAP · STANDARD FOR STANDARD'S SAKE**

'Migrate all five integrations to MCP for consistency' is over-engineering (OE) when three of the five have exactly one client and will never have another. The container revolution paid off at the **pairing** count, not at the container count. One client, one server, no churn → the weld was fine.

⚠️ **TRAP · A LOOKUP IS NOT A CONVERSATION**

The mirror-image error: treating a partner's negotiating *agent* as a stateless tool ('call it like a lookup'). A negotiation crammed into a lookup's shape loses the back-and-forth. When the far side proposes, refuses and counter-offers, brief it, budget it, and verify what it sends back.

The five Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q36 · objective 3.7

Four internal clients (KAVI, control tower, Arjun's IDE, the dispatcher UI) need the same shipment tools, and the tool list grows monthly. The 3.7-correct mechanism:

A. Each client integrates the TMS API directly

✓ **B.** One MCP server; clients discover tools via `tools/list`, so additions propagate without client changes

C. Agent-to-agent between the clients

D. A shared Python library vendored into each client

The tell: four clients + list grows monthly

Why B is right, in simple words: Many clients AND a changing list — both counters are high, so the standard connector earns its keep. Add a tool once; four clients see it next time they ask for the menu.

Why the others fall:

A [OE] four welds, re-welded every month

C [VG] nothing here negotiates; these are plain function calls

D [CM] a copied library in each client re-creates the 'update four places' problem

Dock Book Q37 · objective 3.7

A tool must let clients read SOP 214's full text without spending a tool call. Which MCP primitive, and why?

- A. A tool returning the file contents
- ✓ B. A resource (sop://214): data-not-action, readable into context via resources/read
- C. A prompt embedding the SOP text
- D. An elicitation returning the document

The tell: read ... without spending a tool call

Why B is right, in simple words: Reading a document changes nothing in the world — that is exactly what a resource is for. It is read straight into context, no round-trip.

Why the others fall:

- A [CM] works, but spends the action primitive on a data need
- C [VG] prompts are procedure templates, not document transport
- D [VG] elicitation asks the USER a question; it does not fetch files

Dock Book Q38 · objective 3.7

A workbook states MCP remote servers use the SSE transport and that servers commonly call the client's LLM via sampling. Against the current spec, the accurate correction:

- A. Both statements are current
- ✓ B. Transports are stdio and Streamable HTTP; sampling (and logging) are deprecated — integrate the LLM directly, log to stderr/OTel
- C. SSE is current; sampling is deprecated
- D. Both were removed years ago; the workbook invented them

The tell: against the current spec

Why B is right, in simple words: This is the DRIFT box word for word. Both statements were once true and both moved in the 2026-07-28 spec.

Why the others fall:

- A [VG] both moved
- C [VG] half-updated
- D [VG] they existed; they are deprecated, not fictional

Dock Book Q39 · objective 3.7

IndiaPost offers one REST endpoint; exactly one internal caller will ever use it; the latency budget is tight. The 3.7 choice:

- A. Stand up an MCP server around it now
- ✓ B. Call the API directly in code; wrap it as an MCP tool only if a second client appears
- C. Agent-to-agent with IndiaPost
- D. Refuse the integration as non-standard

The tell: exactly one internal caller ... tight budget

Why B is right, in simple words: One client, one endpoint, no churn: MCP multiplies pairings, and there is only one pairing to multiply. The direct weld is less machinery and one hop faster.

Why the others fall:

- A [OE] standard for standard's sake
- C [VG] there is no agent on the far side, just a window
- D [CM] the data is needed; only the mechanism was in question

Dock Book Q40 · objective 3.7

Delhivery exposes a booking AGENT that negotiates pickup slots given constraints. Integrating it correctly means:

A. Wrapping it as a stateless tool and calling it like a lookup

✓ **B.** Agent-to-agent: send a structured brief (consignment, constraints, authority limits), budget the exchange, and treat replies as proposals to verify

C. Screen-scraping its UI

D. Asking Delhivery for database access instead

The tell: negotiates pickup slots

Why B is right, in simple words: The far side has judgement. You exchange goals and proposals, not function calls: brief it, cap the conversation, and verify what it proposes before acting on it.

Why the others fall:

A [CM] a negotiation crammed into a lookup's shape loses the back-and-forth

C [SD] brittle and unauthorised

D [SD] replaces an interface with an org-chart fantasy

Carry these into the room

SAY IT IN ONE BREATH

1. MCP = the shipping container: one protocol, any client docks with any server; naming `mcp_server_tool`; discovery via `*/list`.
2. Three primitives: **tools act, resources inform, prompts template**. Reading a document = resource, not tool.
3. The 3.7 sentence: **count clients and churn**. Both high → MCP. One client, no churn → direct call. Far side has judgement → agent-to-agent (brief + budget + verify).
4. Transports today: stdio (local, OS identity) and Streamable HTTP (remote, OAuth recommended). SSE transport is gone; sampling and logging are deprecated; elicitation is current.
5. In FastMCP the type hints write the schema and the docstring is the interface the model reads.

Next: Part 2 audits the toolbag — every tool you plug in through this connector rides in the context window on every call, whether it is used or not.

PART 2 OF 8 · OBJECTIVE 3.1

SOFT — SEEN IN CCAR-F, NOW WITH ECONOMICS

The Bloat Audit

Why every tool you give the AI costs money on every single call, even when nobody uses it — and how to trim the toolbelt with logs, not feelings

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- What a 'tool schema' is and why it rides inside every request like luggage
- How to price the toolbelt with a free probe (`count_tokens`) before spending a rupee
- The arithmetic that wins meetings: $1,500 \text{ wasted tokens} \times 40,000 \text{ calls} = 60 \text{ million tokens a day}$
- The two kinds of bloat — too many tools, and lookalike tools that confuse the model
- The trap on the other side: never trim the rare tool whose one use pays for the year

PART 2 · OBJECTIVE 3.1 · TOOL AND AGENT CONFIGURATION — CAPABILITY BLOAT

The bloat audit — every schema rides in every prompt

Arjun's proposal for KAVI lists nine tools. It looks generous. This part shows why 'generous' is a bill paid forty thousand times a day, and how to read the logs to decide what actually belongs on the belt.

↑ THE TOOLBELT THAT DRAGS ON THE FLOOR

A new fitter proudly wears every tool the store sold him. By noon his belt drags; by day two he reaches for the wrong wrench because six look alike. The old fitter carries three tools and finishes first. **The belt is the context window:** every tool's name, description and schema is paid for on every call, used or not — and lookalike tools make the model fumble.

First, what is a 'tool schema' and why does it cost anything?

When you give the AI a tool, you do not just hand it a function. You hand it a **description card**: the tool's name, a sentence saying what it does, and a list of the inputs it takes (the **schema**, written in JSON). The model has to *read* those cards to decide which one to use. And here is the part beginners miss: **the cards are sent along with every single request**, whether or not the model ends up using any tool. Reading costs tokens; tokens cost money and time.

So a tool has two bills. The **token bill**: its schema rides along on every request. The **confusion bill**: if two tools have overlapping descriptions, the model sometimes picks the wrong one. Both are graded on the exam. And a tool is also a door someone can walk through — that is Part 3's business.

One request with the 9-tool belt (tokens the model must read before it answers):



Same request, trimmed belt:



The crimson slice is the belt tax: read on every call, used on almost none. The question itself is the thin blue sliver.
What the model reads before it can answer 'where is KF-2481?'

Word	In plain words	The dock picture	The exam-grade line
Tool schema	The description card for a tool: name + what it does + what inputs it needs, in JSON.	The label sewn on each wrench pouch.	name, description, input_schema; ~250 tokens each; included in every request that lists the tool.
Context window	Everything the model reads in one request: system prompt, tool cards, conversation, question.	The belt — everything you carry, every trip.	Fixed-size input budget; each token costs money and attention; schemas are standing occupants.
Capability bloat	Giving the AI more tools than the work needs.	The dragging belt.	Unused/overlapping tools raise token cost per call and wrong-tool rates; graded as 3.1.
count_tokens	A free API call that tells you how many tokens a request would cost — without running it.	Weighing the belt on the bench before wearing it.	Free probe; pass model + tools + messages; read input_tokens. 'Free probe before paid call'.

Word	In plain words	The dock picture	The exam-grade line
Lookalike tools	Two tools whose descriptions overlap so the model cannot tell them apart.	Six wrenches that look the same.	Raise wrong-tool rate; fix by merging or sharpening descriptions, not by adding a chooser tool.
Load-bearing rare tool	A tool used rarely, but whose one use matters enormously.	The fire extinguisher: never used, never removed.	Trim by logs AND consequence: escalate_to_human stays at zero weekly calls.

The audit is code, not opinion

Arjun does not argue about which tools 'feel useful'. He weighs both belts on the bench with a free probe, then multiplies by the day's traffic. Hand-type it; read the story column as one tale.

THE NEW FITTER'S BELT — 9 tools

get_shipment_status	carried the traffic
find_alternate_carriers	carried the traffic
read_sop	carried the traffic
lookup_order	0 calls last week
book_slot	0 calls last week
get_weather	0 calls last week
convert_currency	0 calls last week
send_sms	0 calls last week
escalate_to_human	0 calls last week

THE OLD FITTER'S BELT — 3 tools

find_alternate_carriers
get_shipment_status
read_sop

schema tokens per call:



2,314

every schema rides along on EVERY request, used or not

schema tokens per call:



811

the logs, not feelings, chose these three

Two belts on the bench. Membership in the trimmed belt was decided by last week's logs.

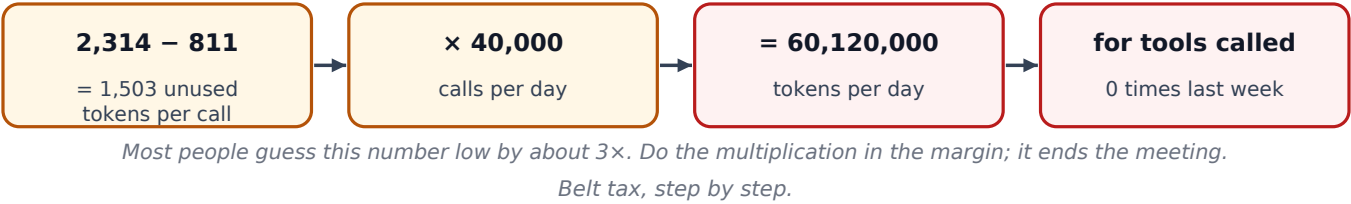
```

e2_bloat_audit.py
1  import json, anthropic
2  client = anthropic.Anthropic()
3
4  def schema_tokens(tools):                # price the belt itself
5      return client.messages.count_tokens( # count_tokens is FREE
6          model="claude-sonnet-5",
7          tools=tools,
8          messages=[{"role": "user", "content": "x"}]).input_tokens
9
10 ALL9 = json.load(open("tools_all9.json")) # the proud new belt
11 CORE3 = [t for t in ALL9 if t["name"] in
12     {"get_shipment_status", "find_alternate_carriers",
13     "read_sop"}] # the old fitter's belt
14
15 bill9, bill3 = schema_tokens(ALL9), schema_tokens(CORE3)
16 print(f"9 tools: {bill9} tokens / call")
17 print(f"3 tools: {bill3} tokens / call")
18 daily = 40_000
19 print(f"belt tax: {(bill9-bill3) * daily:,} tokens/day for tools "
20     f"the logs say were called 0 times last week")

```

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-2	The story: Arjun sets up the bench. anthropic is the library that talks to Claude; cLient is his phone line to it. <i>Example:</i> the weighing scale is plugged in, nothing weighed yet.
4-8	The scale. schema_tokens takes a list of tool cards and asks Claude 'how many tokens would a request carrying these cards cost?' The trick is count_tokens: it prices a request without running it , and it is free. The message is just "x" — a one-token dummy question, so almost the entire count is the tools. <i>Example:</i> put the belt on the scale, read the dial, and nobody has done any work yet.
10-13	Two belts on the bench. ALL9 is the nine-tool proposal loaded from a file. CORE3 keeps only the three tools whose names are in the set — and what decided that set is last week's logs , which say these three carried the traffic. Not anyone's feelings. <i>Example:</i> the old fitter picked his three by looking at what he actually reached for all week. (Part 7's cameras are what make this audit possible at all.)
15-17	Weigh both. Nine tools cost 2,314 tokens per call; three cost 811 . <i>Example:</i> the new belt weighs almost three times the old one, before anyone lifts a wrench.
18-20	The arithmetic that wins meetings. The difference (~1,500 tokens) times 40,000 calls a day is 60,120,000 tokens per day — pure belt tax, for tools nobody called. And that is <i>before</i> counting the wrong-wrench errors. <i>Example:</i> Arjun prints that line and the nine-tool proposal quietly dies.

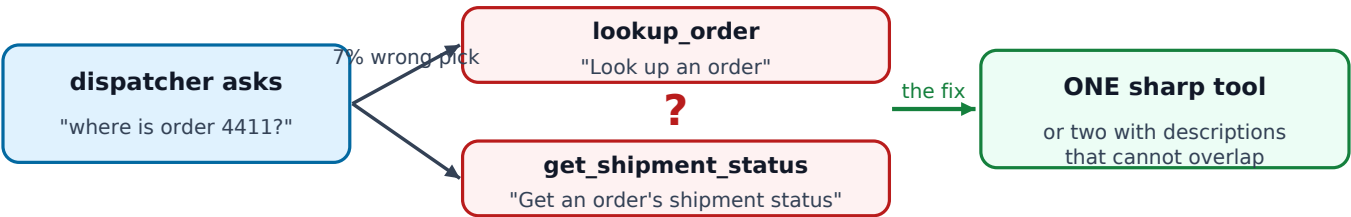
WHAT PRINTS
9 tools: 2,314 tokens / call
3 tools: 811 tokens / call
belt tax: 60,120,000 tokens/day for tools the logs say were called 0 times last week



TRY IT YOURSELF · PREDICT BEFORE RUN
Before running: guess the token cost of ONE tool schema (name + description + JSON schema). Then predict the daily belt tax at 40,000 calls. Write both numbers down first.
Check: one schema is roughly **250-260** tokens. Six unused schemas × 250 × 40,000 ≈ **60 million** tokens/day. If your guess was about a third of that, you are in good company — that is the usual miss.

The subtler bloat: lookalikes

Even a short belt can be bloated if two tools look alike. lookup_order and get_shipment_status overlap enough that the model dials the wrong one 7% of the time. That is a bloat problem too. The fix is **fewer, sharper tools** — merge them, or rewrite both descriptions until the boundary is unambiguous. Not a third 'chooser' tool, not fine-tuning, not routing to the cheaper one.



Six wrenches that look alike → the fitter reaches for the wrong one. Fewer, sharper doors beat a doorman.
Distinct beats many. A belt should have **DISTINCT** tools, not just few.

★ LAW 1 · EVERY TOOL YOU ADD IS A DOOR; KNOW WHO HOLDS THE KEY

A tool is simultaneously a token cost (its schema, every call), a confusion cost (its lookalikes), and a security surface (Part 3). The audit question is never 'might it be useful?' — it is '**who used it last week, and who can walk through it?**'

The trap on the other side — do not trim the fire extinguisher

Once you have learned to trim, the reflex must not over-learn. `escalate_to_human` was called twice last week — both times on refund disputes above ₹10,000, the worst two days of the week. By usage count it is a candidate for the bin. By **consequence** it is the most important tool on the belt: the day it is needed is the day that pays for the year. Trim by logs *and* consequence.

LOGS SAY: USED

LOGS SAY: NOT USED

MISTAKE OR
ABSENCE
IS CHEAP

USED OFTEN · CHEAP IF WRONG

KEEP

the belt's daily bread (status, carriers, SOP)

USED RARELY · CHEAP IF WRONG

TRIM

`get_weather`, `convert_currency` — re-add the day a log line asks

MISTAKE OR
ABSENCE
IS COSTLY

USED OFTEN · COSTLY IF WRONG

KEEP + sharpen

`process_refund`: on the belt, described unambiguously, scoped (Part 3)

USED RARELY · COSTLY IF ABSENT

KEEP

`escalate_to_human`: 2 calls a week — on the worst two days

Trim by logs AND consequence. Usage alone would retire the tool whose rare use is its whole value.

The trim decision as a grid: usage on one axis, consequence on the other.

⚠ TRAP · TRIMMING THE LOAD-BEARING TOOL

When the task genuinely needs a capability — escalation, say, used once a month but on the worst day of the month — usage counts alone do not retire it. An option that says 'keep only the tools with traffic' has missed a constraint (CM). Keep it, describe it sharply, and re-add anything else the day a log line asks.

⚠ TRAP · A MEGA-TOOL IS STILL NINE TOOLS

'Trim to one general-purpose tool that wraps the rest' hides nine schemas inside one worse one: the model now has to guess a sub-command, the description is vaguer, and the token bill barely moves. Over-engineering wearing a slimming costume.

How this connects to the rest of the Dock

- **Part 3 (auth):** every tool on the belt is a door — the fewer doors, the smaller the security audit.
- **Part 7 (observability):** 'called 0 times last week' is only knowable because the logs exist. No cameras, no audit.
- **Part 8 (progressive discovery):** when the belt must be long (30 tools across servers), send a one-line catalogue and fetch full schemas on demand — the belt and the catalogue, combined.

The five Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q1 · objective 3.1

KAVI's config lists nine tools. Last week's logs: three tools carried 99.6% of calls; five were never called; `escalate_to_human` was called twice, both on refund disputes above ₹10,000. Token count shows 2,314 tokens of schema per call versus 811 for a trimmed set. The 3.1-correct trim:

A. Keep all nine — capability is cheap insurance

✓ **B.** Keep the three plus `escalate_to_human`; re-add others when a log line asks

C. Keep only the three with traffic; escalation can be re-added if needed

D. Trim to one general-purpose tool that wraps the rest

The tell: `escalate_to_human` was called twice, both on refund disputes above ₹10,000

Why B is right, in simple words: Usage AND consequence decide. The three carry the traffic; escalation ran twice on the worst days, so its rare use is its whole value. Everything else comes back the day the logs ask for it.

Why the others fall:

A [OE] 1,500 tokens/call of insurance × 40,000 calls/day, plus lookalike confusion

C [CM] retires the tool whose rare use is its entire value

D [OE/VG] a mega-tool hides nine schemas inside one worse one

Dock Book Q2 · objective 3.1

Two tools survive an audit: `lookup_order` and `get_shipment_status`. Logs show the model calls the wrong one in 7% of cases; their descriptions overlap heavily. Best 3.1 fix:

A. Add a third tool that decides which of the two to call

✓ **B.** Merge them, or rewrite both descriptions until the boundary is unambiguous

C. Fine-tune the model on correct tool choices

D. Route all calls to whichever tool is cheaper

The tell: `descriptions overlap heavily`

Why B is right, in simple words: Bloat includes indistinct tools. Fewer, sharper doors beat a doorman. The defect is in the descriptions, so fix the descriptions.

Why the others fall:

A [OE] a coordinator over two tools — the orchestration ghost at tool scale

C [OE] training-scale machinery for a description-scale defect

D [TR] moves the cost dial when the broken dial is distinctness

Dock Book Q3 · objective 3.1

A stem proposes adding six 'likely useful someday' tools before launch, arguing schemas are small. At 40,000 calls/day, 250 tokens per schema, what is the daily cost of that argument, and the verdict?

A. ~0.6M tokens/day; acceptable insurance

✓ **B.** ~60M tokens/day; reject — pay for capability the day a log line asks for it

C. ~6M tokens/day; accept if cached

D. Token cost is irrelevant; reject only if security requires

The tell: $6 \times 250 \times 40,000$

Why B is right, in simple words: $6 \times 250 \times 40,000 = 60,000,000$. Do the multiplication in the margin. 'Someday' tools bill daily.

Why the others fall:

A [VG] slipped two orders of magnitude

C [CM] wrong number, and caching discounts rather than erases; still the wrong default

D [CM] cost is exactly what 3.1 grades, alongside confusion

Dock Book Q4 · objective 3.1

Which situation justifies ADDING a tool this sprint?

- A. The vendor shipped a new API and integration is trivial
- ✓ B. Dispatcher tickets show 40 cases/week manually looking up customs duty rates the model cannot reach
- C. The demo would look stronger with more capabilities
- D. A competitor's agent reportedly has twelve tools

The tell: 40 cases/week ... the model cannot reach

Why B is right, in simple words: The only door-opening argument is a logged, recurring need that the current belt cannot serve. Forty cases a week is a log line asking.

Why the others fall:

- A [OE] ease of adding is not need
- C [SD] demos are not workloads
- D [SD] competitor counts are not constraints

Dock Book Q5 · objective 3.1

KAVI at the agent tier sometimes receives 30+ tools when MCP servers are aggregated. The team proposes exposing every server's full tool list to every request. The measured effect: schema bill triples and wrong-tool calls rise. The 3.1/3.8 answer:

- A. Accept; aggregation is MCP's point
- ✓ B. Expose a per-route subset (the agent tier's own belt), with the rest discoverable on demand
- C. Drop MCP and hardcode the three top tools
- D. Ask the model to ignore irrelevant tools

The tell: schema bill triples and wrong-tool calls rise

Why B is right, in simple words: Bloat control at fleet scale: scope the default belt per route, and let the tail be discovered when needed (Part 8's catalogue).

Why the others fall:

- A [CM] aggregation is transport, not a reason to wear every belt at once
- C [TR] throws away multi-client benefits to fix a config problem
- D [SD] instructions do not unload a belt; config does

Carry these into the room

🗉 SAY IT IN ONE BREATH

1. Every tool schema rides in every request. ~250 tokens each; 9 tools ≈ 2,314 tokens/call; 3 tools ≈ 811.
2. Belt tax = unused schema tokens × daily calls. $1,500 \times 40,000 = 60$ million tokens a day — arithmetic, not opinion.
3. Price the belt with count_tokens (free) before any paid call; let the LOGS pick membership.
4. Two kinds of bloat: too many tools, and lookalike tools. Fix lookalikes by merging or sharpening descriptions.
5. Trim by logs AND consequence: escalate_to_human stays at zero weekly calls. Re-add a tool the day a log line asks.

Next: Part 3 — every tool on the belt is a door. During the audit, one door turns out to open every bay in the building.

PART 3 OF 8 · OBJECTIVE 3.2

NEW TERRITORY — slowly, from zero

The Gate Pass and the Manifest

3

Authentication versus authorization — why 'everyone logs in fine' is exactly where the hole hides, and how a valid key with too many bays on it caused the Refund Heist

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- The difference between 'who are you?' (authN) and 'what may you do?' (authZ), in one dock picture
- What a scope is, what least privilege means, and why a wildcard is a finding, not a convenience
- How to read the little gate-pass code line by line — and why the check lives in code, not in the prompt
- The order that matters: authorize → write the audit log → run the tool, even for denied calls
- The four shapes of 'auth theater' the exam loves, and the six questions on this objective

Source: The Loading Dock — Chapter 3 (Objective 3.2) and Atlas A5. Written for a reader with very little background: story first, words second, code third.

Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F · Days 5–8 of the 35-day CCAR-P campaign · exam 2 Nov 2026

PART 3 · OBJECTIVE 3.2 · AUTHENTICATION AND AUTHORIZATION — IDENTIFY SECURITY GAPS

The gate pass — who you are is not which bays you may enter

This is new territory, so it goes slowly. Everything at Kaveri logs in perfectly. That sentence is the trap: the exam's security questions almost always hide a *permission* gap behind a working *login* story.

↑ THE GATE PASS AND THE MANIFEST

At the dock gate, the guard checks two different papers. Your **gate pass** proves who you are — that is **authentication (authN)**. The **manifest** says which bays your truck may enter — that is **authorization (authZ)**. The heist crews of this world are rarely stopped by the first paper: they walk in with a real pass whose manifest someone forgot to narrow. The Refund Heist's ghost is exactly this: a valid credential with too many bays on it.

Two questions the guard asks, and why they are different

Think of a bank. Showing your ID card at the door proves *who you are*. It does not, by itself, let you open someone else's locker. Those are two separate decisions, made by two separate rules. Software copies this exactly. **Authentication** answers 'is this really the KAVI agent / the refund desk / the carrier portal?' using API keys, OAuth tokens, SSO sessions or service accounts.

Authorization answers 'given that it is the carrier portal, may it process a refund?' using **scopes** — named permissions attached to the identity.

PAPER 1 · THE GATE PASS

authentication (authN) — WHO ARE YOU?

```
KAVERI FREIGHT · GATE PASS
name:  carrier-portal-v1
type:  service account (a machine)
proof: static token, 18 months old
status: VALID ✓ (it logs in fine)
```

API keys · OAuth tokens · SSO sessions · service accounts

PAPER 2 · THE MANIFEST

authorization (authZ) — WHICH BAYS MAY YOU ENTER?

```
MANIFEST for carrier-portal-v1
bay: status:read      ✓ fine
bay: * (WILDCARD)    x every bay!
    → every customer's consignments
    → refunds included
```

scopes · least privilege · per-tool checks in code

Paper 1 is perfect. Paper 2 is the problem. Every 3.2 question is some version of this picture.

Word	In plain words	The dock picture	The exam-grade line
authN (authentication)	Proving who you are.	The gate pass.	Verifies identity: API keys, OAuth tokens on Streamable HTTP, SSO for humans, service accounts for machines.
authZ (authorization)	What that identity is allowed to do.	The manifest — which bays.	Grants actions via scopes; least privilege; checked per tool, in code, before the tool body runs.
Scope	One named permission on a credential, e.g. refund:write.	One bay listed on the manifest.	The unit of authZ; a tool names the ONE scope it requires; the check is set membership.
Least privilege	Every caller gets the minimum permissions that do the job — nothing extra.	No master keys on the keyring.	Design rule; violations (wildcards, god-mode service accounts) are the over-permissive (OP) ghost's Professional costume.

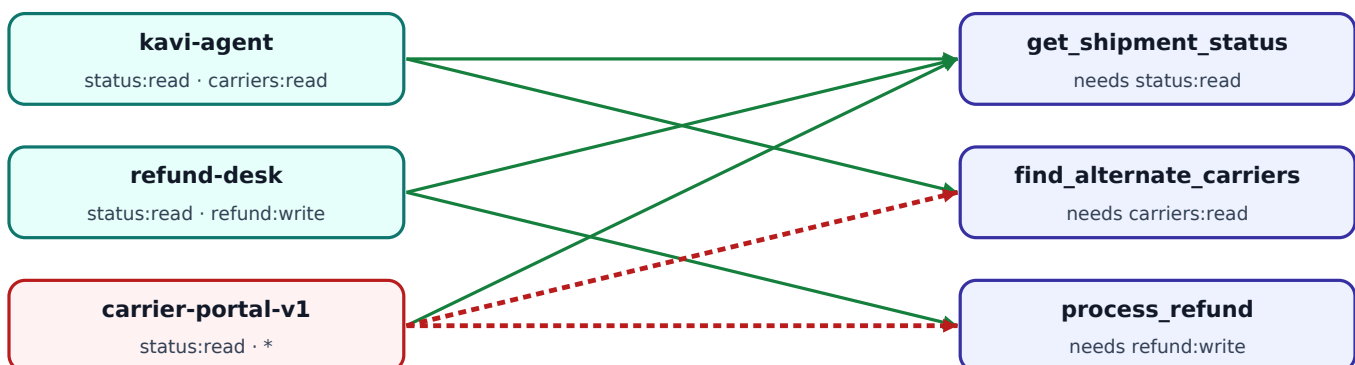
Word	In plain words	The dock picture	The exam-grade line
Wildcard (*)	A scope that means 'everything'.	A key that opens every bay.	A finding, not a convenience. Always issued 'to get the demo working'; always the heist's door.
Token (auth sense)	A string that proves identity/permissions. NOT an LLM token.	The gate pass itself.	Bearer credential on requests (OAuth-issued on Streamable HTTP). Same word, two exam meanings — disambiguate by context.
Service account	An identity for a machine, not a person.	The forklift's own gate pass.	Per-service accounts keep audit trails honest; one shared account destroys attribution.
Audit log	An append-only record of who did what, when — including refusals.	The gatehouse ledger.	Written on every sensitive call INCLUDING denials; feeds incident review and the denials-by-caller dashboard (Part 7).
OAuth	A standard way for a program to get a token with specific scopes, and to rotate it.	The office that issues time-limited passes.	MCP's recommended route for remote (Streamable HTTP) servers; per-team tokens, scoped, rotated.

The code — the scoped key, checked in code

Below is the whole security model of the control tower in 25 lines. That smallness is the point: what fits on one page can be reviewed in a meeting; what cannot be tabled cannot be audited. Hand-type it; read the story column as one tale.

CALLER (identity) + scopes held

DOOR (tool) — the ONE scope it needs



■ allowed by an explicit scope ■ allowed only because of the wildcard — the ghost

Callers, their scopes, and the doors. Line 4's wildcard is the crimson dashed pair.

e3_gate_pass.py

```

1  SCOPES = {                                # the manifest, in code
2      "kavi-agent":      {"status:read", "carriers:read"},
3      "refund-desk":     {"status:read", "refund:write"},
4      "carrier-portal-v1": {"status:read", "*"},    # <- the ghost: wildcard
5  }
6  REQUIRED = {                                # every door names its key
7      "get_shipment_status": "status:read",
8      "find_alternate_carriers": "carriers:read",
9      "process_refund":      "refund:write",
10 }
11
12 def authorize(caller: str, tool: str) -> None:

```

```

13     granted = SCOPES.get(caller, set())
14     needed = REQUIRED[tool]                # unknown tool -> KeyError: good
15     if needed not in granted and "*" not in granted:
16         raise PermissionError(
17             f"{caller} lacks {needed} for {tool}")    # structured, loggable
18
19     def run_tool(caller, tool, args):
20         authorize(caller, tool)                # BEFORE the tool, always
21         audit_log(caller=caller, tool=tool, args=args) # Part 7 will read this
22         return TOOLS[tool](**args)
23
24     # the fix, after the audit:
25     SCOPES["carrier-portal-v1"] = {"status:read"}    # wildcard revoked

```

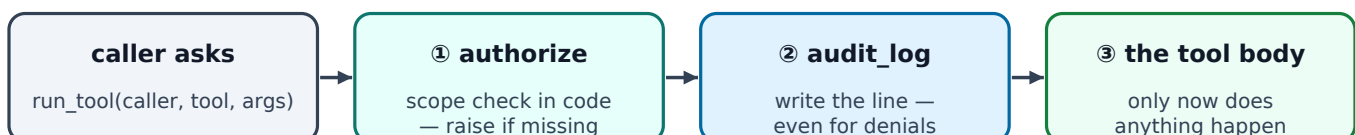
Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-5	The story: Saturday's audit. Arjun writes the manifest as a plain dictionary: each caller, and the set of bays it may enter. <i>Example:</i> the KAVI agent may read statuses and carrier lists — and nothing else. Line 4 is the ghost : eighteen months ago someone gave the carrier portal a * 'to get the demo working'. It authenticates perfectly. Your exam job is reading credential tables like this one, not login flows.
6-10	Every door names its one key. process_refund needs refund:write; nothing else opens it. This tiny table IS the security model — small enough to review in a meeting, which is precisely the point. <i>Example:</i> a laminated sheet at the gate: 'Bay 7 — manifest must say refund:write'.
12-17	The check. Look up what the caller holds (unknown caller → empty set → denied). Look up what the door needs (unknown tool → a crash, which is <i>good</i> : a door nobody listed must not open). Then plain set membership: if the needed scope is not held — and the caller has no wildcard — raise a named, loggable error. Note what this is not : a sentence in the system prompt saying 'only the refund desk may refund'. CCAR-F's law — config beats instruction — is now called authZ and graded at Professional weight. <i>Example:</i> the guard does not ask the driver nicely; he reads the manifest and either lifts the barrier or writes 'DENIED: lacks refund:write'.
19-22	The order of operations is the lesson. authorize first — before anything happens. Then audit_log — the ledger line is written even for denied calls (a denied call raised on line 20, and in the real code the log is written in both branches). Only then the tool body. Tomorrow's dashboard has a 'denials by caller' panel; a spike on it at 03:00 is how you catch a probe in progress.
24-25	The fix is one line because the model was scopes-in-a-table. The heist replay now dies at the gate with a named reason. Least privilege is not a slogan; it is a small diff.

WHAT PRINTS

```

kavi-agent -> process_refund      PermissionError: lacks refund:write
refund-desk -> process_refund      OK (scope matches)
carrier-portal-v1 -> process_refund OK <- silent, valid, catastrophic
# after line 25: PermissionError. The audit closed the bay.

```



*Order is the lesson. A denied call never reaches ③ — but it always reaches ②, which is how you notice a probe.
authorize → audit_log → tool. Say it in that order; the exam checks the order.*

 **TRY IT YOURSELF · PREDICT BEFORE RUN**

Predict the three outcomes before running: kavi-agent calling process_refund; refund-desk calling process_refund; carrier-portal-v1 calling process_refund BEFORE line 25 runs. The third answer is the whole chapter.

Check: denied (lacks refund:write) · allowed (scope matches) · **allowed** — silent, valid, catastrophic. The portal never needed refunds; it had them anyway, for eighteen months, and nothing 'bad' happened. That is not safety; that is luck with a ledger.

 **SABOTAGE LAB · BREAK EXACTLY ONE THING**

Change line 15 to if needed not in granted: (drop the wildcard clause). Predict which caller's behaviour changes and in which direction. Run all three probes. **Lesson:** a security check you did not test both ways is a guess — and yes, this sabotage makes the system SAFER; notice that feeling and log it.

Where the pass gets checked depends on the road

Recall Part 1's two transports. A **local stdio server** inherits the identity of whoever launched it — the gate is the operating system; there is no separate pass to show. A **remote Streamable HTTP server** authenticates the standard HTTP way: bearer tokens, API keys, and **OAuth is MCP's recommended route** for obtaining them. So: **authN is transport-shaped; authZ is yours either way**. The scope table exists whichever road the caller came in on.

Caller (identity)	authN — proves itself by	Scopes held (the manifest)	May therefore touch
KAVI agent (service acct)	API key, rotated	status:read · carriers:read · sop:read	TMS status, carrier lists, SOP store
Refund desk service	OAuth token (Streamable HTTP)	status:read · refund:write	process_refund — the ₹10k human gate still applies above threshold
Dispatcher UI	SSO session	status:read · draft:write	read statuses, edit drafts — never refunds
carrier-portal-v1 (AS FOUND)	static token, 18 months old	status:read · * (wildcard)	everything — every customer's consignments, refunds included
carrier-portal-v1 (FIXED)	OAuth, 90-day rotation	status:read only	statuses on its own lane, nothing else
Observability shipper	mTLS service identity	logs:write	append to the log store — cannot read business data

Rules this table encodes: per-caller identities (no shared god account) · every tool names its required scope in code · denials are logged and dashboarded · wildcards are findings, not conveniences · the gate check runs before the tool body, always.

 **TRAP · THE MAP NOBODY DREW**

The heist survives in systems where this table exists only in people's heads. An exam option that says 'document and review per-caller scopes' beats clever cryptography options whenever the question shows a working login and an unexplained data leak.

Auth theater — four shapes that sound diligent and secure nothing

A prompt that forbids

"Never refund from the UI"
— while the tool list still offers process_refund

→ the model can be talked past a sentence

A wildcard 'for now'

scope * issued 18 months ago "to get the demo working"

→ a valid key to every bay

One god-mode account

users log in — then every tool runs as one shared service account

→ the log reads 'somebody' on every line

Denials nobody reads

authorization decisions logged to a channel with no dashboard

→ the probe at 03:00 scrolls past

In rising order of exam frequency. The tell in 3.2 stems is a credential that WORKS — then ask what it works ON.

A simple way to spot theater: ask **'if the model were fully convinced it should do the wrong thing, what stops it?'** A sentence in the prompt does not — the model can be persuaded past a sentence. A missing tool and a denied scope do — the action is *impossible*, not merely discouraged. **Config beats instruction.**

The six Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q6 · objective 3.2

Every integration authenticates flawlessly. During an audit Arjun finds the carrier-portal token, issued 18 months ago, carries a wildcard scope. Nothing bad has happened. The 3.2-correct reading:

A. No incident, no problem; schedule a review next quarter

✓ **B.** A live security gap: valid authN with over-broad authZ is exactly how the Refund Heist worked — narrow it now

C. Rotate the token; rotation fixes staleness

D. Add a system-prompt rule forbidding misuse of the portal

The tell: authenticates flawlessly ... carries a wildcard scope

Why B is right, in simple words: Paper 1 is perfect; paper 2 lists every bay. That is the gap, and it is live today, incident or not.

Why the others fall:

A [SF] the discovery IS the incident; waiting prices the next heist

C [CM] a fresh key to every bay is still a key to every bay

D [SD] persuasion guarding a door the config left open

Dock Book Q7 · objective 3.2

Where must the check 'refund-desk may call process_refund; nobody else may' live?

- A. In the system prompt, stated firmly
- ✓ B. In code, before the tool body runs, keyed on the caller's scopes
- C. In the model's judgement, given good context
- D. In weekly log review

The tell: nobody else may

Why B is right, in simple words: 'Nobody else may' is a guarantee, and guarantees live in code. Config beats instruction — authorization is its Professional name.

Why the others fall:

- A [SD] the persuasion gate; CCAR-F's core discriminator
- C [SD] judgement is not a control
- D [SF] detects Tuesday's breach on Friday

Dock Book Q8 · objective 3.2

The team runs every tool call through one shared service account 'to simplify ops.' Which capability dies, and why does the exam care?

- A. Latency budgeting; extra hops
- ✓ B. Attribution: the audit log can no longer say WHO did what, so authZ cannot be scoped per caller and incidents cannot be traced
- C. Caching; accounts affect cache keys
- D. Nothing dies; simplification is good ops

The tell: one shared service account

Why B is right, in simple words: One identity for all callers means the ledger reads 'somebody' on every line. Per-caller identity is the precondition for scopes, audits and honest dashboards.

Why the others fall:

- A [VG] unrelated mechanism
- C [VG] unrelated mechanism
- D [CM] the simplification deletes least privilege AND auditability

Dock Book Q9 · objective 3.2

A remote MCP server for the SOP store must be exposed to three internal teams over Streamable HTTP. The 3.2-grade authN choice:

- A. A single long-lived API key shared by the teams
- ✓ B. OAuth-issued tokens per team, scoped sop:read, with rotation
- C. IP allowlisting alone; the network is trusted
- D. No auth; SOPs are internal documents anyway

The tell: remote ... three internal teams over Streamable HTTP

Why B is right, in simple words: Remote transport + multiple callers → standard HTTP auth, OAuth recommended, one scoped token per team, rotated.

Why the others fall:

- A [OP] a shared key = no attribution, no narrow scopes
- C [SD] networks move; identity is not an IP address
- D [CM] 'internal' data still feeds prompts; poisoned or leaked context is a real gap

Dock Book Q10 · objective 3.2

Head-to-head. The stem: 'dispatcher UI users must never trigger refunds, even if the model is convinced.' (B) omit refund tools from the UI's tool set and deny refund:write to its identity; (C) instruct the model that dispatcher sessions must not refund, and log attempts. Pick and justify.

A. C — instructions plus logging is defence in depth

✓ **B.** B — capability absent and scope denied: the action is impossible, not discouraged

C. Either; both prevent refunds

D. C, but with a louder instruction and alerts

The tell: even if the model is convinced

Why B is right, in simple words: 'Even if the model is convinced' names model conviction — which only config survives. No tool + no scope = impossible.

Why the others fall:

A [SD] logging an attempt is not preventing it

C [CM] one prevents, one requests

D [SD] volume is not enforcement

Dock Book Q11 · objective 3.2

Which log discipline turns the gate pass system into a detection system?

A. Log successful calls only; denials are noise

✓ **B.** Log every authorization decision including denials, and dashboard denials by caller

C. Log nothing sensitive; privacy first

D. Log full request bodies for everything

The tell: detection system

Why B is right, in simple words: The audit log is a sensor. Denied calls are its most interesting lines — a spike of denials from one caller is a probe in progress.

Why the others fall:

A [SF] the probe pattern lives in the denials

C [CM] decision metadata is not payload; auditability is a security control

D [OP] payload hoarding creates a new leak to guard

Carry these into the room

SAY IT IN ONE BREATH

1. authN = who you are (gate pass). authZ = which bays (manifest). The heist ghost is a VALID credential with a wildcard manifest.
2. Scope check in code, BEFORE the tool body; per-caller identities; least privilege; wildcards are findings.
3. Order: authorize → audit_log → tool. Log every decision INCLUDING denials; dashboard denials-by-caller — a 03:00 spike with healthy latency is a probe.
4. stdio inherits the OS user; Streamable HTTP wants OAuth + scopes. AuthN is transport-shaped; authZ is yours either way.
5. 'Never do X' = omit the tool and deny the scope, not a louder prompt. Config beats instruction.

Next: Part 4 leaves the gate and walks into the warehouse: 400 SOP documents must become answerable, and how you shelve them decides whether Padma gets the right paragraph.

PART 4 OF 8 · OBJECTIVE 3.5

NEW TERRITORY — the week's most expensive lesson

Shelving the SOPs — Chunking and Indexing

4

How 400 rulebooks become answerable: cut them into boxes along their meaning, label every box from the inside, and file them in two registers — and why a lazy cut gave Padma the wrong paragraph

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- What RAG is, in one dock picture — and the first question to ask before building one at all (is the corpus small enough to skip it?)
- Why cutting documents by length instead of by meaning is how a cancellation procedure answers a re-filing question
- The three ways to chunk SOP 214, line by line — fixed, by heading, with overlap — and what each buys and bills
- What an embedding, a vector index and BM25 are, and why 'contextual retrieval' writes a label inside every box
- The numbers that decide: 35% → 49% → 67%, retrieve 150 → pass 20, corpus under ~200k tokens → no RAG

PART 4 · OBJECTIVE 3.5 · RAG PIPELINE DESIGN — CHUNKING AND INDEXING

Shelving the SOPs — chunking is where retrieval is won or lost

Monday, 11:40. Padma is on the phone. KAVI just read her the steps for *cancelling* an e-way bill when she asked how to *re-file* one. She now has a cancelled bill. Nothing was wrong with the AI. Something was wrong with how the rulebooks were cut into pieces — six weeks earlier, before any question was ever asked.

📦 HOW YOU SHELVE THE BOXES

A warehouse that saws every incoming shipment into equal half-metre boxes has beautiful, uniform shelves — and a sofa in six boxes on four aisles. When the order for the sofa comes: three trips and a missing armrest. The rival warehouse shelves along the seams: one product, one box, one label. **Chunking is shelving:** cut along the document's meaning — its headings, its sections — not along a tape measure.

First, what is RAG and why does Kaveri need it?

Kaveri has 400 Standard Operating Procedures (SOPs) — rulebooks, in markdown, updated weekly, some in mixed Hindi-English. A dispatcher asks KAVI 'how do I re-file an e-way bill?'. The answer is in SOP 214, section 4. The AI has never read SOP 214; it has to be *handed* the right paragraph at question time. That is **RAG — Retrieval-Augmented Generation**: **retrieve** the right pieces of text, then let the model **generate** an answer from them. Three verbs, three jobs: **shelve** them (chunk), **file** them (index), **find** them (retrieve). This part is shelving and filing. Part 5 is finding.

🕒 ASK THIS FIRST

The pre-question, before any RAG design: is the corpus big enough to need RAG at all? Under roughly **200,000 tokens** (about 500 pages), stable and always relevant — skip RAG. Put the whole thing in the prompt behind prompt caching. The 400 SOPs are ~1.4 million tokens, so Kaveri needs RAG. A 38k-token rate card does not (that is Dock Book Q26).

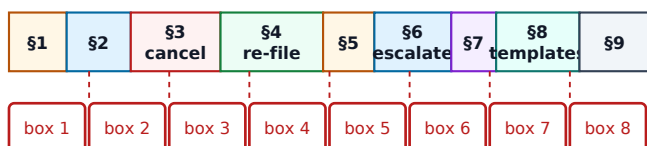
Word	In plain words	The dock picture	The exam-grade line
RAG	Retrieve the right text, then generate an answer from it.	Find the right box, then read from it.	Retrieval-Augmented Generation: chunk → index → retrieve → prompt assembly → generate. Skip it when the corpus is ≤ 200k tokens.
Corpus	The whole collection of documents you want answerable.	Everything in the warehouse.	Kaveri: 400 SOPs, ~1.4M tokens, heading-structured, updated weekly.
Chunking	Cutting documents into retrievable pieces.	Shelving: one meaning, one box.	Splitting a corpus into units for indexing; strategy (fixed / semantic / heading-aware / overlap) sets retrieval's ceiling.
Chunk	One piece — the unit that gets found and handed to the model.	One box on the shelf.	Should match query granularity: dispatchers ask section-level questions → section-level chunks.
Overlap	Each chunk also carries the tail of the previous one.	A photo of the neighbour taped inside the box.	Seam insurance; costs duplicate tokens indexed and sometimes retrieved twice — a dial tuned per corpus.

Word	In plain words	The dock picture	The exam-grade line
Embedding	A text turned into a list of numbers that captures its meaning.	The foreman's memory of an aisle.	Dense vector; similar meanings land near each other in vector space; produced by a paid embedding model.
Vector index	The database that stores embeddings for fast 'nearest neighbour' search.	The foreman, institutionalised.	Answers 'which chunks are nearest this query vector' at scale.
BM25 / keyword index	Classic exact-word search with clever weighting.	The register: literal, synonym-blind.	Lexical ranking rewarding term frequency and rarity; FIRST-CLASS in current practice, not legacy.
Contextual retrieval	Each chunk gets a sentence saying where it is from, before indexing.	A label INSIDE every box.	Prepend chunk-situating context before embedding AND BM25; failure-rate cuts 35% → 49% → 67% as stages stack.
Ingest	The one-time (or weekly) job of chunking and indexing.	Receiving day at the warehouse.	Runs when the corpus changes; scheduled re-chunk + re-index with version labels inside chunks.

The incident, dissected

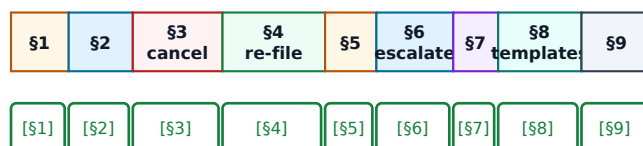
Arjun's first RAG build cut all 400 SOPs into **fixed 500-token chunks** — simple, uniform, meaning-blind. The query 'e-way bill re-filing' retrieved a chunk that starts mid-sentence in §3 (bill *cancellation*) and ends before §4's actual procedure. Both sections say 'e-way bill' on every line, so the retriever could not tell them apart. KAVI confidently read out the cancellation steps. Here is the same document shelved two ways:

TAPE-MEASURE SHELIVING — every 500 words, cut



▲ box 3 starts mid-§3 and ends before §4's steps

SEAM SHELIVING — one section, one box



▲ nine boxes; §4 re-filing is whole, and its label is written INSIDE

Same document. One shelving can answer 'how do I re-file?' in one retrieval; the other cannot.

The tape measure versus the seams. Retrieval quality was decided here, at ingest — before any embedding model got a vote.

The code — chunking three ways on SOP 214

One real document, three shelving policies, the same retrieval question at the end. Hand-type it; the story column is one tale.

e4_chunking.py

```

1  import re
2  SOP = open("sops/sop_214.md", encoding="utf-8").read()
3
4  def fixed(text, size=500):
5      words = text.split()
6      return [" ".join(words[i:i+size]) for i in range(0, len(words), size)]
7
8  def by_heading(text):
9      parts = re.split(r"(?m)^(#{1,3} .+)$", text) # keep the headings
10     chunks, title = [], "SOP 214"
11     for p in parts:
12         if p.startswith("#"): title = p.strip("# ").strip()

```

```

13         elif p.strip():
14             chunks.append(f"[SOP 214 - {title}]\n{p.strip()}")
15         return chunks
16
17     def with_overlap(chunks, tail=40):          # a photo of the neighbour
18         out = []
19         for i, c in enumerate(chunks):
20             prev_tail = " ".join(chunks[i-1].split()[-tail:]) if i else ""
21             out.append((prev_tail + "\n" + c).strip())
22         return out
23
24     for name, ch in [("fixed", fixed(SOP)), ("heading", by_heading(SOP))]:
25         hit = [c for c in ch if "re-filing" in c and "Step 1" in c]
26         print(f"{name:8s}: {len(ch):3d} chunks | full procedure in one chunk: "
27               f"{bool(hit)}")

```

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-2	The story: re-shelving the warehouse that hurt Padma. Read SOP 214 (about 4,000 words, nine headed sections) into one string called SOP. <i>Example:</i> the whole rulebook lying flat on the receiving bench.
4-6	The tape measure. Split into words, then take 500 at a time: words 0-499 become box 1, 500-999 box 2, and so on. Simple, uniform — and meaning-blind . §3 and §4 end up sharing a box, which is exactly how a cancellation procedure answers a re-filing question. <i>Example:</i> the saw goes through the sofa at the half-metre mark whatever is there. Fixed-size is not evil; it is the default that must be argued <i>for</i> , not <i>from</i> .
8-9	The seams. The regex finds every markdown heading (#, ##, ###) at the start of a line, and — because of the capturing brackets — keeps the headings in the resulting list instead of throwing them away. <i>Example:</i> the receiving clerk cuts exactly where the manufacturer printed a dotted line.
10-15	The quiet trick that carries the whole chapter. Walk the pieces. When a piece is a heading, remember it as title. When a piece is body text, save it with its title written into the chunk text itself : [SOP 214 - §4 Re-filing] followed by the paragraph. A chunk that knows where it came from embeds better, keyword-matches better, and cites itself. <i>Example:</i> a label on the <i>inside</i> of the box, so whoever opens it knows it is the §4 sofa even if it was carried to the wrong aisle. Hold this thought: Part 5's contextual retrieval is this trick grown up.
17-22	The photo of the neighbour. For each chunk, take the last 40 words of the previous chunk and glue them on the front. A sentence that straddles a seam now exists whole in at least one box. <i>Example:</i> box §4 opens with the last two lines of §3 — if the re-filing rule began 'as above, but...', the 'above' is right there. Overlap costs duplicate tokens: a dial, not a default.
24-27	The verdict, printed. For each policy: how many boxes, and does any single box contain both the phrase 're-filing' and 'Step 1' (i.e. the whole procedure)? Fixed: 8 boxes, False . Heading: 9 boxes, True . Same document, same question — one policy can answer it in one retrieval, one cannot.

WHAT PRINTS

```

fixed    :   8 chunks | full procedure in one chunk: False
heading  :   9 chunks | full procedure in one chunk: True
# the fixed cut lands mid-#3; "e-way bill" appears in both #3 and #4,
# so the retriever cannot tell cancellation from re-filing. The heading
# cut carries "[SOP 214 - §4 Re-filing]" INSIDE the chunk text.

```

[§3 Cancellation] ...cancel the bill ...

last 40
words of
prev

[§4 Re-filing] Step 1 ... Step 4

last 40
words of
prev

[§5 Fees] ...

Overlap = a photo of the neighbour: a sentence straddling a seam exists whole in at least one box. Costs duplicate tokens — a dial, not a default.

Overlap, drawn.

✎ TRY IT YOURSELF · PREDICT BEFORE RUN

Predict both printed lines before running: chunk counts (SOP 214 is ~4,000 words with 9 headed sections) and, for each strategy, whether the complete re-filing procedure lands inside ONE chunk. Then predict which strategy retrieves the CANCELLATION section for the re-filing query, and why the two sections' shared vocabulary makes that easy.

Check: $4,000 \div 500 = 8$ fixed boxes; 9 headed sections = 9 heading boxes. Only the heading cut keeps §4 whole. Fixed retrieves §3 because both sections repeat 'e-way bill' and the box that scores highest happens to start in §3.

✂ SABOTAGE LAB · BREAK EXACTLY ONE THING

Change line 9's regex to match only ^# (top-level headings). Predict the new chunk count and whether the §4 chunk is now too big to be a precise answer. **Lesson:** chunk granularity should match query granularity — dispatchers ask section-level questions, so section-level boxes. One giant box per document is 'context is king' and it kills precision (Q23's option C).

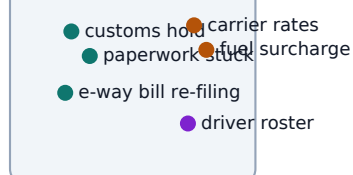
★ LAW 2 · CHUNK ALONG MEANING, NOT LENGTH

Cut on the document's own structure (headings, sections, tickets) and write each chunk's origin into its text. Size and overlap are dials tuned per corpus; the seam is the principle. **A retrieval system's ceiling is set at the shelving stage — everything downstream can only lose height.**

Filing the boxes — two registers, and a label inside every box

Once the boxes are cut, they need to be *findable*. Two very different filing systems exist, and the industry now uses both at once (Part 5 explains the finding; here is what gets built at ingest).

THE FOREMAN'S MAP OF MEANING



An embedding turns a text into a list of numbers — a position on this map. Texts that MEAN similar things land near each other, even with zero shared words ('paperwork stuck' sits next to 'customs hold'). The vector index is the map with a fast 'what is nearest this point?' lookup. Cosine similarity is the ruler: how near are two points.

The foreman files by meaning. The register (BM25) files by exact words. Build both.

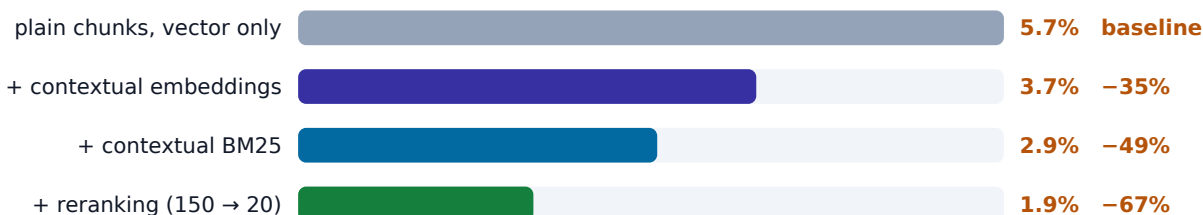
The register (BM25) is a keyword index: ask for 'e-way bill' and it returns every box whose text contains those words, and nothing else. Literal, exact, synonym-blind. **The foreman (embeddings + vector index)** files by meaning: ask for 'customs paperwork problem' and he points at the right aisle although no label says those words. Fuzzy, and sometimes points at the plausible-but-wrong aisle.

⇌ DRIFT · THE DOCS MOVED

What older teaching says: Chunk → embed → vector search → top-5 into the prompt; keyword search is the legacy thing RAG replaced.

What the current docs say (checked Sep 2026): Anthropic's current guidance (**Contextual Retrieval**): prepend a chunk-situating sentence before BOTH embedding and BM25 — keyword search is a first-class half, not legacy. Measured on top-20 retrieval failure rate: contextual embeddings alone cut it **35%** (5.7 → 3.7%), plus contextual BM25 **49%** (→ 2.9%), plus reranking **67%** (→ 1.9%). The reranking shape: retrieve ~150 candidates, rerank, pass top-20 — and top-20 beat top-5 and top-10 outright. And the pre-question: under ~200k tokens of corpus, skip RAG and cache the whole thing.

Top-20 retrieval FAILURE rate (lower is better) — Anthropic's contextual-retrieval measurements



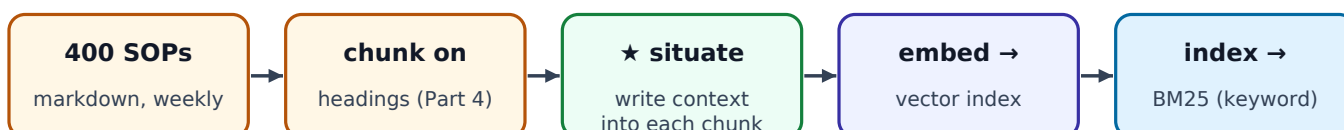
Each stage stacks on the previous. Reranking shape: retrieve ~150 candidates → rerank → pass top-20 (20 beat 5 and 10 outright).

The numbers to keep. Say them aloud: 35 → 49 → 67; 150 → 20; 200k.

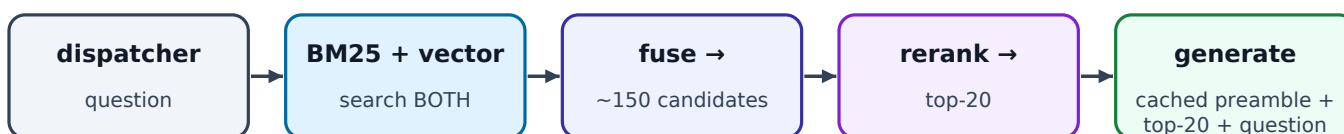
E4's little habit — writing [SOP 214 – §4] into each chunk — is contextual retrieval's baby form. The full technique has a **model write a situating sentence per chunk at ingest** (once, cacheable, paid one time): 'This chunk is from the customs-hold SOP and describes how to re-file an e-way bill after a hold.' Both the foreman and the register then see that sentence instead of a bare paragraph. Same principle, industrial grade — and it is why the query 'customs paperwork stuck' can find §4 even though §4 never says 'stuck'.

The whole pipeline, drawn

INGEST LANE · runs WEEKLY (the SOPs change)



QUERY LANE · runs 40,000 TIMES A DAY

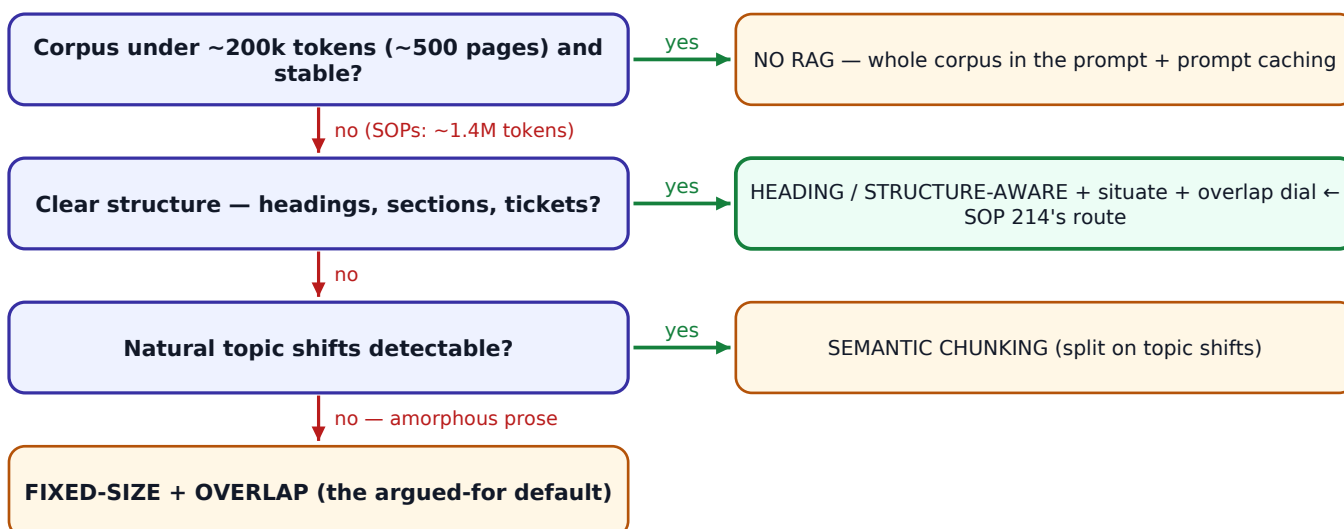


The star is contextual retrieval: context is written into each chunk BEFORE both indexes see it. Answers cite the chunk labels.

Atlas A1: two lanes. The top one runs weekly; the bottom one runs 40,000 times a day.

Weekly updates. Indexes are caches of the corpus; when SOPs change, the index must follow. Schedule a re-chunk + re-index run, and write **version labels inside the chunks** so a stale citation is visible ('SOP 214 v7, Aug 2026'). Without that, the model will fluently cite the repealed procedure and nobody will know.

Which chunking strategy? Atlas A2 as a walk



Overlay dials on every route: chunk size $\approx$ query granularity · overlap = seam insurance · situating context in every chunk · re-chunk only when the miss pattern says the *SHELVING* failed (right document, wrong section).
Document shape in, strategy out. SOP 214's route is the green row.

⚠ TRAP · RE-CHUNKING AS A CURE FOR EVERYTHING

Once burned, teams re-chunk at every retrieval miss. Diagnose by **where the right paragraph fell out of the pipeline**: 'wrong section' → chunking (shelving) · 'right document, wrong rank' → fusion/reranking (Part 5) · 'right chunk retrieved, wrong answer written' → prompt assembly. Fix the stage that dropped it.

⚠ TRAP · ONE CHUNK PER DOCUMENT — 'CONTEXT IS KING'

A 900-token box answering a 40-token question: precision dies, the model reads eight irrelevant sections for every relevant one, and top-20 becomes 18,000 tokens. Granularity must mirror the questions.

The seven Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q23 · objective 3.5

The SOPs are heading-structured, updated weekly, queried at section granularity. The 3.5-correct chunking:

- A. Fixed 500-token chunks; uniform is efficient
- ✓ B. Heading-aware chunks with the section title written into each chunk's text, modest overlap
- C. One chunk per document; context is king
- D. Sentence-level chunks for precision

The tell: heading-structured ... queried at section granularity

Why B is right, in simple words: Structure exists and the questions are section-shaped — cut on the seams, label each box from the inside, add a little overlap.

Why the others fail:

- A [CM] meaning-blind cuts split procedures (the KF-2481 incident)
- C [TR] 900-token answers to 40-token questions; precision dies
- D [CM] granularity finer than any question; procedures shredded

Dock Book Q24 · objective 3.5

Retrieval for 'e-way bill re-filing' returns the CANCELLATION section. Both sections share vocabulary. Chunks were cut fixed-size mid-section. Where is the fix?

A. Swap the embedding model

✓ B. Re-chunk on headings so each section is one self-labelled unit — the failure happened at ingest

C. Raise top_k so both sections come back

D. Add a system-prompt warning about cancellation vs re-filing

The tell: cut fixed-size mid-section

Why B is right, in simple words: The miss pattern says 'wrong section' — the ceiling was set at shelving time, so fix shelving.

Why the others fall:

A [TR] a better foreman cannot find a box that was sawn in half

C [SF] ships the confusion to the model and hopes

D [SD] persuasion downstream of a structural defect

Dock Book Q25 · objective 3.5

What does contextual retrieval add over plain heading-aware chunking, and what does it cost?

A. Nothing measurable; it is branding

✓ B. A model-written situating sentence per chunk before embedding AND BM25 — one-time ingest cost (cacheable), for a 35–49% retrieval-failure cut before reranking

C. A second embedding model at query time — doubling query cost

D. Automatic re-chunking on every query

The tell: before embedding AND BM25 ... one-time

Why B is right, in simple words: Situate at ingest; both retrieval channels benefit; the bill is paid once.

Why the others fall:

A [VG] the numbers are published and large

C [VG] the cost lands at ingest, not per query

D [VG] nothing re-chunks at query time

Dock Book Q26 · objective 3.5

The corpus is the carrier rate card plus 6 SOP templates: 38k tokens, changes quarterly, needed on almost every call. The 3.5-correct architecture:

A. A full RAG pipeline with hybrid retrieval

✓ B. No RAG: include it all in the prompt behind prompt caching

C. Fine-tune the model on the rate card

D. A vector index over the templates only

The tell: 38k tokens, changes quarterly, needed on almost every call

Why B is right, in simple words: 38k tokens, stable, always used — far under the ~200k skip-RAG threshold. The pre-question answers itself.

Why the others fall:

A [OE] retrieval machinery for a corpus that fits in the prompt

C [OE] training-scale solution; and rate cards change quarterly

D [CM] half-machinery for a corpus needing none

Dock Book Q27 · objective 3.5

Weekly SOP updates arrive. Which pipeline property is REQUIRED for retrieval to stay truthful?

- A. Real-time re-embedding on every edit
- ✓ B. A scheduled re-chunk + re-index run, with version labels inside chunks so stale citations are visible
- C. Nothing; embeddings age gracefully
- D. Doubling top_k to compensate for staleness

The tell: updated weekly ... stay truthful

Why B is right, in simple words: Indexes are caches of the corpus; caches need an invalidation story, and staleness must be visible.

Why the others fall:

- A [OE] per-edit pipelines for weekly-cadence docs
- C [SF] the model will fluently cite the repealed procedure
- D [TR] more stale chunks are not fresher chunks

Dock Book Q28 · objective 3.5

Overlap of 40 tokens is proposed for the heading-aware chunks. The honest accounting:

- A. Overlap is free insurance
- ✓ B. Seam-straddling sentences survive in one whole copy; cost = duplicate tokens indexed and sometimes retrieved twice — a dial, tuned per corpus
- C. Overlap fixes vocabulary confusion between sections
- D. Overlap replaces the need for situating context

The tell: honest accounting

Why B is right, in simple words: 3.5 grades knowing what each ingest dial buys AND bills. Both sides named.

Why the others fall:

- A [VG] nothing indexed twice is free
- C [VG] a different failure mode entirely
- D [VG] orthogonal techniques

Dock Book Q29 · objective 3.5

Head-to-head for the 400-SOP corpus: (B) contextual chunks + hybrid + rerank (answer p95 +200 ms, failure rate 1.9%); (C) plain chunks + vector-only (p95 unchanged, failure rate 5.7%). The stem's only constraint: dispatcher answers must cite the correct section $\geq 97\%$ of the time; latency budget 4 s, current p95 2.8 s. Pick.

- A. C — never add latency
- ✓ B. B — $98.1\% \geq 97\%$ while C's 94.3% fails the floor; +200 ms fits inside 4 s
- C. C, with top_k raised to compensate
- D. B, but only if latency is removed from the SLA

The tell: $\geq 97\%$... 4 s, current p95 2.8 s

Why B is right, in simple words: Bind the constraint first (the 97% floor): $100 - 1.9 = 98.1\%$ clears it; $100 - 5.7 = 94.3\%$ does not. Then check the other currency has room: $2.8 + 0.2 = 3.0 \text{ s} < 4 \text{ s}$.

Why the others fall:

- A [TR] guards slack latency, breaches the binding accuracy floor
- C [SF] more of the wrong shortlist is not a corrected shortlist
- D [CM] nothing needs removing; $3.0 \text{ s} < 4 \text{ s}$

Carry these into the room

SAY IT IN ONE BREATH

1. Pre-question: corpus $\leq$ 200k tokens (~500 pages), stable, always used → NO RAG; whole corpus + prompt caching.
2. Law 2: chunk along meaning — cut on headings, write the origin INTO the chunk, size $\approx$ query granularity; overlap is a priced dial.
3. Build TWO indexes: BM25 (the register, exact words) and embeddings in a vector index (the foreman, meaning). Keyword is first-class, not legacy.
4. Contextual retrieval = a situating sentence per chunk, once at ingest, before BOTH indexes. Failure-rate cuts 35% → 49% → 67% (5.7 → 3.7 → 2.9 → 1.9%). Retrieve ~150, rerank, pass 20.
5. Weekly corpus → scheduled re-index + version labels inside chunks. Diagnose misses by stage: wrong section = chunking · wrong rank = fusion/rerank · wrong answer from right chunk = prompt assembly.

Next: Part 5 — the boxes are shelved and filed. Now three different questions arrive at the desk, and each needs a completely different machine to answer it.

PART 5 OF 8 · OBJECTIVE 3.6

NEW TERRITORY — the matrix you redraw from memory

Three Questions, Three Routes — Retrieval by Shape

5

The register and the foreman working together (hybrid search), and why the most expensive mistake in this domain is feeding a lookup or a count into a similarity search

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- Why keyword search and meaning search each fail alone — on different questions, silently — and why the industry asks both
- The hybrid retriever, line by line: cosine similarity, a simplified BM25, one weight that blends them, a rerank on top
- The three question shapes — an ID, a count, a procedure — and the three machines that answer them
- The retrieval-by-shape matrix (Atlas A3): rows are what the data IS, columns are what the question ASKS
- Why a fluent wrong number is worse than an error, and the six exam questions on this objective

Source: The Loading Dock — Chapters 5 and 6 (Objective 3.6), Atlas A3. Written for a reader with very little background: story first, words second, code third.

Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F · Days 5–8 of the 35-day CCAR-P campaign · exam 2 Nov 2026

PART 5 · OBJECTIVE 3.6 · RETRIEVAL STRATEGY BY DATA SHAPE AND QUERY

The register, the foreman, and the accountant — retrieval follows the shape of the data

Part 4 shelved the boxes and built two filing systems. This part is about *finding*: how the two systems are asked together, and — the bigger lesson — how to notice when a question should not go to the shelves at all.

↑ ASK THE OLD FOREMAN AND CHECK THE REGISTER

Two ways to find a box in the warehouse. The **register** lists exact words — ask for 'e-way bill' and it returns every box whose label contains those words, and nothing else. The **old foreman** has walked these aisles thirty years; ask for 'customs paperwork problem' and he points at the right aisle even though no label says those words — he files by meaning. The register is literal and misses synonyms; the foreman is fuzzy and sometimes points at the plausible-but-wrong aisle. The trick the whole industry converged on: **ask both, trust where they agree**.

Two finders, two blind spots

THE REGISTER · keyword search (BM25)

"Every box whose label contains the words 'e-way bill'.
Nothing else."

✓ wins when:

the query quotes exact strings: codes ('HSN 8471'),
ids ('KF-'), section numbers, the exact phrase
're-filing'

✗ fails when:

the question uses different words from the document:
'parcel stuck, customer furious' — no SOP says
'stuck'. Zero shared words → zero results.
Synonym-blind.

THE FOREMAN · semantic (embeddings)

"Customs paperwork problem? Aisle 4, third shelf." —
no label says those words; he files by meaning.

✓ wins when:

the question is a symptom described in the user's
own words; synonyms and paraphrases; Hindi-English
mixes

✗ fails when:

the question is an exact code or phrase: 'HSN 8471'
embeds near 'HSN 8473'; the plausible-but-wrong
aisle. Approximate by nature.

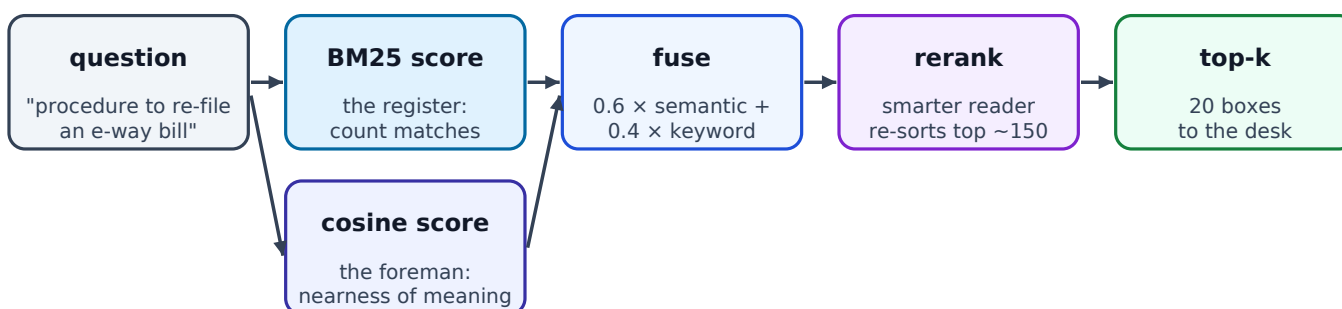
Names for the furniture: register = BM25 keyword search; foreman = semantic search over embeddings.

Word	In plain words	The dock picture	The exam-grade line
Semantic search	Finding text by meaning, using embeddings.	Asking the foreman.	Query embedded → nearest chunk vectors by cosine similarity. Bridges synonyms; approximate on exact strings.
Keyword search (BM25)	Finding text by exact words, with clever weighting.	Checking the register.	Lexical ranking rewarding term frequency and rarity; exact, literal, synonym-blind; first-class, not legacy.
Cosine similarity	A number saying how close two meaning-vectors point.	The foreman's 'how near is this aisle'.	Angle between two vectors; you never compute it in the exam — you say what it measures: closeness of meaning, not sharing of words.
Hybrid search	Keyword and semantic search fused into one ranking.	Ask both; trust where they agree.	Score fusion of BM25 + embeddings; one weight leans it; each channel covers the other's blind queries.

Word	In plain words	The dock picture	The exam-grade line
Reranking	A second, smarter pass that re-sorts the shortlist.	The supervisor re-sorting the picks.	A cross-reading model scores query+chunk pairs; retrieve ~150 → rerank → top-20; ~+200 ms for a 67% failure cut.
top_k	How many chunks the retriever hands the model.	How many boxes reach the desk.	Candidate count passed onward; Anthropic's tests: 20 beat 5 and 10 for final answering; k=3 for quick lookups.
Recall vs precision	Did we fetch ALL the right boxes vs ONLY right boxes.	Missing the sofa vs bringing the whole aisle.	recall = fraction of relevant chunks found; precision = fraction of found chunks relevant; dials trade them.
Structured lookup	Fetching THE answer from a database by its key.	Reading the bay number off the tracking board.	For ids and keyed rows; exact, not approximate; no RAG.
Aggregate query	Counting, summing or averaging over rows.	The accountant adding up the ledger.	SQL over the tables; for 'how many / average / trend / per day' language. RAG cannot count.
Context poisoning	Bad retrieved text steering the answer.	A mislabelled box on the desk.	Irrelevant or adversarial retrieved content degrading generation; mitigations: better retrieval, provenance labels, scoped prompts.

The code — the hybrid, small enough to hand-type

Everything is stubbed so the plumbing runs free in DRY_RUN. The story column is one tale: the warehouse gets both a register and a foreman, and learns to ask both.



Ask both; trust where they agree. Each half fails alone, on different queries, silently — which is why the failure hides in demos.

The query lane, zoomed in. Lines 20-27 are the fuse box.

e5_hybrid.py norm() scales keyword scores to 0-1; CHUNKS from Part 4

```

1  import math, os
2  from collections import Counter
3  DRY_RUN = os.environ.get("DRY_RUN") == "1"
4
5  def embed(text):
6      if DRY_RUN:
7          return [hash(w) % 97 / 97 for w in sorted(set(text.lower().split()))][:8]
8      return real_embed(text)
9
10 def cosine(a, b):
11     n = min(len(a), len(b))
12     dot = sum(x*y for x, y in zip(a[:n], b[:n]))
13     return dot / (math.sqrt(sum(x*x for x in a[:n]) or 1) *
14                  math.sqrt(sum(y*y for y in b[:n]) or 1))
15
16 def bm25ish(q, chunk):

```

```

17     words = Counter(chunk.lower().split())
18     return sum(words[w] for w in q.lower().split())
19
20 def hybrid(query, chunks, k=3, w_sem=0.6):
21     qv = embed(query)
22     scored = []
23     for c in chunks:
24         s_sem = cosine(qv, embed(c))           # the foreman's opinion
25         s_kw = bm25ish(query, c)               # the register's count
26         scored.append((w_sem * s_sem + (1 - w_sem) * norm(s_kw), c))
27     return [c for _, c in sorted(scored, reverse=True)[:k]]
28
29 top = hybrid("procedure to re-file an e-way bill", CHUNKS)
30 print([c.splitlines()[0] for c in top])      # the shortlist's labels

```

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-3	The story: the bench is set up with a free switch. If the environment variable DRY_RUN is 1, nothing paid happens. <i>Example:</i> a practice run of the warehouse with cardboard boxes before the real stock arrives.
5-8	A pretend foreman for practice. In DRY_RUN, embed makes a fake-but-deterministic list of 8 numbers from the words of the text (hash each word, squash into 0-1). Same text → same numbers, every time, for free. In real life line 8 calls a paid embedding model. <i>Example:</i> the trainee foreman who has memorised the aisle list but not the meaning — good enough to test that the plumbing works.
10-14	The foreman's ruler. Cosine similarity: multiply the two vectors element by element and add up (the dot product), then divide by their lengths so only the <i>direction</i> counts. Result near 1 = same direction = similar meaning; near 0 = unrelated. The or 1 prevents dividing by zero. You never compute this in the exam — you say what it measures: closeness of meaning, not sharing of words.
16-18	The register, honestly simplified. Count how many times each query word appears in the chunk, add them up. Real BM25 also rewards rare words and damps long documents, but the property that matters is here: literal, exact, synonym-blind . 'stuck' scores zero against a chunk that says 'hold'.
20-27	The fusion. Embed the question once. For every chunk, get the foreman's opinion (cosine) and the register's count (normalised to 0-1), and blend them with one weight: w_sem=0.6 means 60% meaning, 40% words. Sort highest first, hand back the top k boxes. Each question shape leans on a different half — which is the entire argument for hybrid: each half fails alone, on different queries, silently .
29-30	Printing labels, not full chunks. Because Part 4 wrote origins into the text, the first line of every chunk is its label — the shortlist is self-describing. Small habits from the shelving chapter keep paying.

WHAT PRINTS

```

q1: ['[SOP 214 - S4 Re-filing]', '[SOP 214 - S3 Cancellation]', ...]
    # exact phrase -> the register (keyword) half carried it
q2: ['[SOP 214 - S1 What a customs hold is]', '[SOP 214 - S4 Re-filing]', ...]
    # zero keyword overlap -> the foreman (semantic) half carried it
# kill either half (sabotage) and one of the two queries fails.

```

🔧 TRY IT YOURSELF · PREDICT BEFORE RUN

Two queries, predict each shortlist's top label BEFORE running: (1) 'e-way bill re-filing' — which half of the hybrid does the work when the exact phrase exists in the corpus? (2) 'customs paperwork stuck, what do I do' — no SOP contains the word 'stuck'; which half rescues it?

Check: (1) the register — exact phrase, literal match. (2) the foreman — zero shared words, shared meaning.

✂ SABOTAGE LAB · BREAK EXACTLY ONE THING

Set `w_sem=1.0` on line 26 (kill the register). Predict which of the two probe queries degrades and HOW — wrong section, or right section at a lower rank? Run, ledger the result, restore.

Lesson: single-channel retrieval fails on a query *shape*, not at random — which is why the failure hides in demos that only ever ask one kind of question.

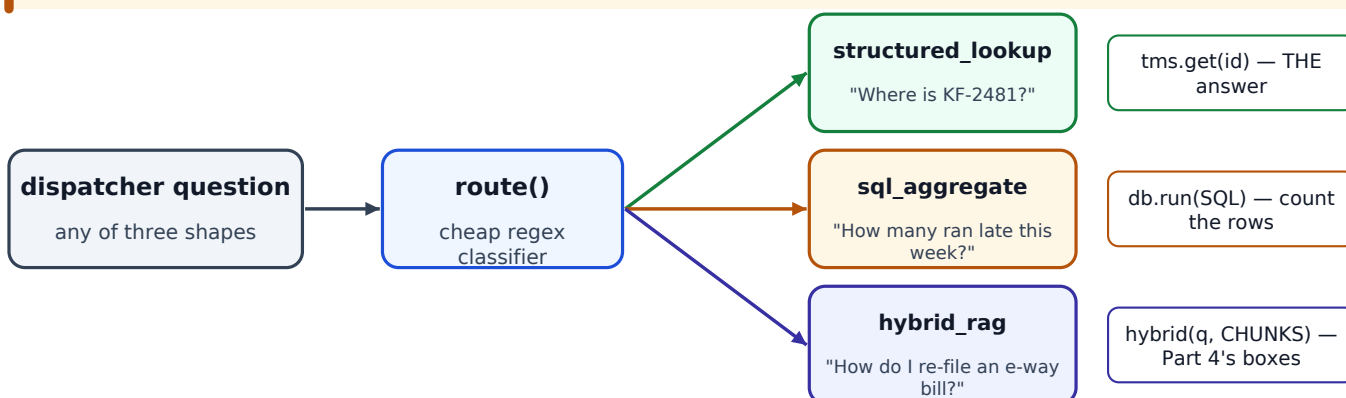
Tuning the blend by the question's shape

The one weight on line 26 is a dial, and the question's shape sets it. Queries that quote **exact codes** ('HSN 8471', 'KF-', section numbers) want the fusion leaned toward **keyword/BM25** — exact strings are the register's home game. Queries that describe a **symptom in the user's own words** ('customer says the parcel is stuck and she's furious') want it leaned toward **semantic** — the SOPs say 'customs hold' and 'escalation', not 'stuck' and 'furious', and only the foreman bridges that vocabulary gap. And on top of the fused shortlist sits the **reranker**: retrieve ~150 candidates cheaply, let a slower, smarter reader re-sort them, pass the top 20 — about +200 ms for a 67% cut in failures.

Three questions, three routes — the biggest lesson in this domain

↑ YOU DON'T ASK THE FOREMAN FOR BAY NUMBERS

'Where is consignment KF-2481?' — you look it up in the tracking system; asking the wise old foreman to reminisce about it would be absurd. 'How do I re-file an e-way bill?' — now you want the foreman and the register. 'How many shipments ran late this week?' — neither: that is the **accountant's** job, a sum over rows. Three question shapes, three completely different machines — and the most expensive mistake in this domain is feeding a lookup or a sum into a similarity search.



A cheap classifier at the front; specialist machinery behind. This is D1's routing pattern one level down the stack.

e6_shape_router.py

```

1  import re
2
3  def route(query: str) -> str:
4      if re.search(r"\bKF-\d{4,}\b", query):          # an ID -> exact world
5          return "structured_lookup"
6      if re.search(r"how many|count|average|trend|this week|per day",
7                  query, re.I):                      # aggregate language
8          return "sql_aggregate"
9      return "hybrid_rag"                            # policy/procedure prose
10
11  HANDLERS = {

```

```

12  "structured_lookup": lambda q: tms.get(extract_id(q)), # no RAG!
13  "sql_aggregate":      lambda q: db.run(to_sql(q)),      # count rows
14  "hybrid_rag":         lambda q: answer_from(hybrid(q, CHUNKS)),
15  }
16
17  for q in ["Where is KF-2481 right now?",
18           "How many shipments ran late this week?",
19           "What is the procedure to re-file an e-way bill?"]:
20      print(f"{route(q):18s} <- {q}")

```

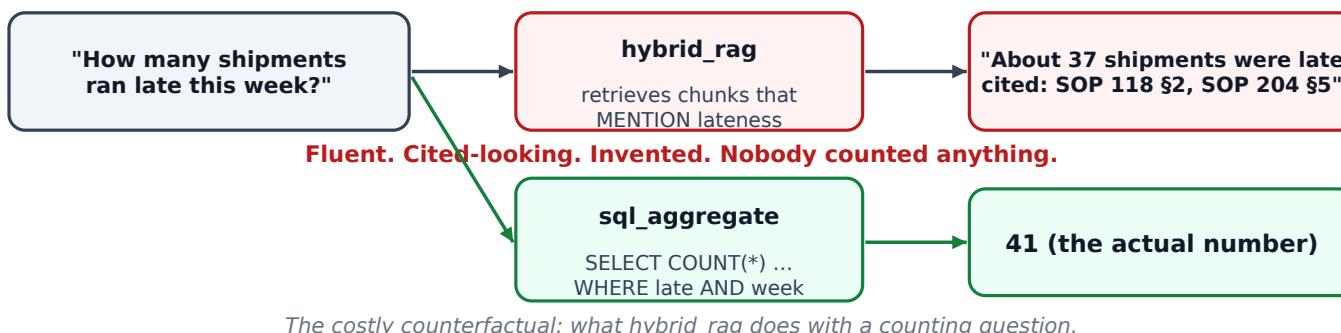
Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-9	The story: one desk, three question shapes, a router at the front. route looks for a consignment id pattern (KF- followed by four or more digits) → the exact world. Then for counting words ('how many', 'average', 'trend', 'this week') → the accountant. Anything else is policy/procedure prose → the shelves. Cheap classification first, specialist machinery second. <i>Example:</i> the receptionist glances at the question and points you to the tracking board, the accounts office, or the library.
11-15	Three handlers, three worlds. The comment on line 12 is the chapter's thesis: an ID lookup is exact-answer work — the tracking database holds THE answer, and similarity search can only return things LIKE the answer. Semantic machinery on exact questions does not merely waste money; it converts certainty into probability . Line 13 counts rows with SQL. Only line 14 uses Part 4's boxes.
17-20	The three probes, one per shape. Say the mapping aloud, it is exam-recitable: ID or code → structured lookup · count/trend → aggregate query · policy/procedure prose → hybrid RAG · exact phrase or code snippet → keyword-weighted · vague symptom prose → semantic-weighted.

WHAT PRINTS

```

structured_lookup <- Where is KF-2481 right now?
sql_aggregate     <- How many shipments ran late this week?
hybrid_rag        <- What is the procedure to re-file an e-way bill?
# hybrid_rag on the count question retrieves chunks that MENTION lateness
# and the model composes a plausible number. Fluent, cited-looking, wrong.

```



TRY IT YOURSELF · PREDICT BEFORE RUN

Predict all three printed routes. Then the costly counterfactual, in the notebook: what does hybrid_rag return for 'How many shipments ran late this week?' — and why is a fluent wrong number worse than an error?

Check: an error stops the dispatcher and makes her ask a human. A fluent, cited-looking '37' goes straight into the Tuesday report. Silent failures are the worst class because nobody knows to look.

★ **LAW 3 · RETRIEVAL FOLLOWS THE SHAPE OF THE DATA, NOT THE FASHION**

Structured data wants lookups. Countable data wants aggregates. Prose wants hybrid search — keyword-leaning when queries quote exact terms, semantic-leaning when they describe symptoms. The fashionable answer (embed everything) is the over-engineering of this domain; **the stem's data shape is the tell that picks the machine.**

The retrieval-by-shape matrix — Atlas A3, redraw it from memory on Day D

Rows are what the data IS; columns are what the question ASKS. The crimson cells are the expensive mistakes; every 3.6 question lives somewhere on this grid.

DATA SHAPE ↓ · QUESTION →	Exact entity "where is KF-2481"	Procedure / policy "how do I re-file"	Symptom prose "stuck, what now"	Count / trend "how many late"
Rows with keys (TMS / WMS)	Structured LOOKUP — the answer, not a guess	RAG here = OE — join the SOP corpus instead	Semantic search of rows = fluent nonsense	SQL AGGREGATE
Structured docs (400 SOPs)	Keyword-weighted hybrid (codes, section numbers)	Hybrid + rerank, top-20	Semantic-leaning hybrid (synonyms carry it)	RAG cannot count — index metadata, aggregate that
Free-text logs / emails	Keyword (ids quoted verbatim)	Hybrid; situate chunks with ticket metadata	Semantic-leaning hybrid	Not retrieval — log pipeline → metrics (Part 7)
Tiny stable reference (rate card)	No retrieval: in-context + cache	No retrieval: in-context + cache	Same	Trivial in code

the right machine keyword-lean semantic-lean no retrieval the expensive mistake

Read a 3.6 stem twice: once for the row, once for the column. Excellent machinery in the wrong cell is still wrong.

◎ **THE TELL**

Read a 3.6 stem twice: once for the **row** (what the data IS), once for the **column** (what the question ASKS). An option that is excellent machinery in the wrong cell is still wrong — the grid outranks the craft.

⚠ **TRAP · RAG AS A HAMMER**

The 3.6 distractor of record: 'index the TMS records so the model can semantically search shipment statuses.' The TMS already answers *exactly*; an index answers *approximately*. When a stem's data is rows with keys, the RAG option is wrong however well-designed its chunking is. Save your RAG craft for the 400 SOPs, where it is the right machine.

⚠ **TRAP · REPLACING THE ROUTER WITH AN AGENT**

When the cheap router misroutes 2% of aggregate-language questions, the fix is its rule set (or a small classifier) with those 2% as an evaluation set — not an agent that 'reasons about each query'. A known-category decision at 40,000 a day is workflow territory (D1 Law 2); an agent loop there is latency and cost to re-solve a solved classification.

The six Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q30 · objective 3.6

'Where is KF-2481 right now?' The system semantically searches an index of TMS records and answers from the most similar chunks. The 3.6 verdict:

A. Good design; everything benefits from embeddings

✓ **B.** Wrong machine: keyed rows answer exactly via lookup; similarity search converts certainty into probability

C. Acceptable if recall is tuned high

D. Acceptable if the index refreshes hourly

The tell: semantically searches an index of TMS records

Why B is right, in simple words: Rows with keys + an exact-entity question — the matrix's most crimson cell. IDs → lookup, always.

Why the others fall:

A [OE] the fashionable hammer

C [VG] recall tuning cannot make similarity exact

D [CM] freshness does not fix approximateness

Dock Book Q31 · objective 3.6

'How many shipments ran late this week?' routed to hybrid RAG returns a fluent, cited-looking number. What actually happened, and the fix?

A. The RAG counted the rows; fine

✓ **B.** Chunks MENTIONING lateness were retrieved and the model composed a plausible number — route count/trend language to an aggregate query instead

C. top_k was too low for counting

D. The embedding model is weak at numbers

The tell: fluent, cited-looking number

Why B is right, in simple words: Retrieval fetches text ABOUT lateness; only aggregation counts it. RAG cannot count.

Why the others fall:

A [SF] fluent wrong numbers are the worst failure class

C [VG] no k counts rows

D [VG] not an embedding property

Dock Book Q32 · objective 3.6

Queries quoting exact codes ('HSN 8471', 'KF-', section numbers) underperform on the semantic-heavy hybrid. The 3.6 dial:

A. Raise semantic weight further

✓ **B.** Lean the fusion toward keyword/BM25 for code-bearing queries — exact strings are the register's home game

C. Strip codes from queries before search

D. Lower top_k

The tell: exact codes ... underperform on the semantic-heavy hybrid

Why B is right, in simple words: Query shape tunes the fusion: codes → keyword-leaning; symptoms → semantic-leaning.

Why the others fall:

A [TR] leans away from what the failing queries need

C [CM] deletes the strongest signal in the query

D [CM] depth is not channel balance

Dock Book Q33 · objective 3.6

'Customer says the parcel is stuck and she's furious, what do I do' retrieves poorly on keyword search alone. Why, and the fix?

A. The corpus lacks relevant SOPs

✓ **B.** No lexical overlap: SOPs say 'customs hold' and 'escalation', not 'stuck' and 'furious' — the semantic channel bridges the vocabulary gap

C. The query is too long for BM25

D. Add the word 'stuck' to all SOPs

The tell: stuck ... furious ... keyword search alone

Why B is right, in simple words: Symptom prose versus procedure vocabulary — zero shared words, shared meaning. Semantic search exists precisely for the synonym gap.

Why the others fall:

A [CM] SOP 214 answers it, under other words

C [VG] length is not the mechanism

D [SD] vandalising the corpus to meet one query

Dock Book Q34 · objective 3.6

Free-text carrier emails must become searchable for dispute resolution. The 3.6-grade ingest:

A. Embed raw emails as-is

✓ **B.** Situate each email chunk with metadata (carrier, consignment, date, thread) written into the text, then hybrid-index

C. Keyword-index only; emails are informal

D. Summarise each email and index only summaries

The tell: free-text ... dispute resolution

Why B is right, in simple words: Free text with rich metadata — write the metadata INTO the chunks so both channels see it. Situating context is how messy corpora become retrievable.

Why the others fall:

A [CM] raw chunks strand ids and dates outside both indexes

C [CM] informal text is exactly where synonyms rule

D [SF] summaries drop the disputed sentence you will need verbatim

Dock Book Q35 · objective 3.6

A D1 crossover. The retrieval router (E6) misclassifies ~2% of aggregate-language queries into hybrid RAG. The team proposes replacing the router with an agent that reasons about each query. Verdict:

A. Adopt; agents route better

✓ **B.** Keep the router; fix its rule set or classifier with the 2% as an eval set — a known-category decision is workflow territory (D1 Law 2)

C. Run both and compare vibes

D. Route everything to hybrid RAG for simplicity

The tell: known-category decision ... 2% as an eval set

Why B is right, in simple words: Known categories, a cheap decision, 40,000 a day — D1's routing signature, not an agent's. Domains compose: D1 picks the pattern, D3 picks the retrieval machine behind it.

Why the others fail:

A [OE] loop latency and cost to re-solve a solved classification

C [SD] 'compare' without a metric or set

D [CM] reinstates the fluent-wrong-number failure

Carry these into the room

SAY IT IN ONE BREATH

1. Hybrid: keyword misses synonyms, semantic misses exact codes — each fails alone, silently, by query shape. Ask both; trust where they agree.
2. Codes / exact phrases → keyword-lean. Symptom prose → semantic-lean. Rerank on top: ~150 → 20, +200 ms, -67% failures.
3. Law 3: ID → lookup · count/trend → aggregate · procedure prose → hybrid RAG. RAG CANNOT COUNT.
4. Similarity search on exact questions converts certainty into probability; a fluent wrong number is worse than an error.
5. The A3 grid: read the stem for the row (data shape) and the column (question shape). Great machinery in the wrong cell is still wrong.

Next: Part 6 — every choice so far had a price in speed or accuracy. The dials chapter is about buying the right one on purpose, and writing down why.

PART 6 OF 8 · OBJECTIVE 3.3

SOFT — SEEN BEFORE, NOW YOU MUST JUSTIFY WITH NUMBERS

The Dials — Accuracy and Latency Are Bought, Not Wished

Every setting in the system trades the same two currencies. The skill is not choosing a speed — it is routing each truck to the right speed and writing down why, in one sentence with a number in it

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- What latency, accuracy, an SLA, a p95 and a cost ceiling actually mean, in plain words
- The six dials Kaveri turns — model tier, top-k, reranking, context size, timeouts, parallel fan-out — and what each buys and bills
- How to read a question: underline the currency it FIXES, and every dial then has only one legal direction
- The shape of an exam-grade justification: constraint + dial + direction + number
- The two traps: spending the currency the question fixed, and 'justifying' with adjectives

Source: The Loading Dock — Chapter 7 (Objective 3.3). Written for a reader with very little background: story first, words second, code third.

Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F · Days 5-8 of the 35-day CCAR-P campaign · exam 2 Nov 2026

PART 6 · OBJECTIVE 3.3 · ACCURACY-LATENCY TRADE-OFFS, JUSTIFIED

The dials — accuracy and latency are bought, not wished

Parts 4 and 5 kept saying 'this costs 200 ms' and 'this cuts failures 67%'. This part is about making those choices on purpose: which dial, which direction, and the one sentence — with a number in it — that justifies the move.

↑ THE EXPRESS LANE AND THE INSPECTION BAY

Every dock runs two speeds. The **express lane** waves routine trucks through in seconds — a glance, a stamp. The **inspection bay** takes twenty minutes and catches what the glance cannot. A dock that inspects everything jams; a dock that inspects nothing gets robbed. **The skill is not choosing a speed — it is routing each truck to the right speed and writing down why.**

Two speeds, one dock

THE EXPRESS LANE

seconds — a glance, a stamp

- routine trucks: status lookups, delay drafts
- small model (Haiku-class router), k=3, no reranker
- 2-second SLA on this path
- an error here costs about ₹4

THE INSPECTION BAY

twenty minutes — catches what the glance cannot

- the expensive trucks: refunds, SOP answers, agent-tier investigations
- large model (Opus-class), k=20, reranker ON (+200 ms)
- latency budget slack; accuracy floor binding
- an error here costs about ₹1,800

A dock that inspects everything jams; a dock that inspects nothing gets robbed. Route each truck to the right speed.

The words, in plain language

Word	In plain words	The dock picture	The exam-grade line
Latency	How long the user waits for an answer.	Minutes at the gate.	End-to-end time per request; measured per path; promised at the tail (p95), not the average.
Accuracy	How often the answer is right (or cites the right section).	Trucks inspected correctly.	Measured on an eval set; has a cost per error (₹4 for a delay draft, ₹1,800 for a wrong refund).
SLA	A promise about speed: 'answers within 2 seconds'.	The sign at the gate: 'trucks cleared in 5 minutes'.	Service-level agreement; a promise about p95/p99, so a healthy mean can still breach it.
p95	The time that 95% of requests beat — i.e. what the unlucky 1 in 20 experiences.	The queue as the unlucky see it.	95th-percentile latency; SLAs live here; Part 7 explains why the mean lies.
Cost ceiling	The most you may spend per exception / per call, in rupees.	The budget per truck.	e.g. ₹6 blended per exception; a fixed currency that constrains model tier and context size.
Accuracy floor	The least accuracy you may accept on a path.	'No refund goes out with less than a full inspection'.	e.g. cite the correct section ≥ 97%; a fixed currency that constrains cheaper options.

Word	In plain words	The dock picture	The exam-grade line
Model tier	Small / medium / large model — cheaper and faster vs smarter and dearer.	Junior clerk, senior clerk, the inspector.	Haiku-class for routing, Sonnet-class for drafts, Opus-class where errors are expensive.
top-k	How many retrieved chunks are handed to the model.	Boxes brought to the desk.	k=20 after reranking for SOP answers (measured best); k=3 for quick lookups quoting one section.
Timeout / retry / degrade	Give up on a slow dependency after N seconds, try once more, then fall back to something honest.	If the carrier does not pick up in 3 rings, ring once more, then read the last-known ETA off the board.	A hung integration must cost a known maximum; degrade with a staleness label, never an unknown wait.
Parallel fan-out	Ask several things at the same time instead of one after another.	Three scouts sent out together.	Wall clock = slowest scout, not the sum; the SLA buys concurrency, not model speed.
Prompt caching	Re-using a large fixed prefix so it is read at ~10% of the price.	The laminated preamble on every desk.	Cache reads ~90% off, above a per-tier minimum cacheable size (1,024–4,096 tokens).

Every dial trades the same two currencies



**The question FIXES one currency (an SLA, a cost ceiling, an accuracy floor).
Underline it. Then every dial has exactly one legal direction.**

Accuracy on one side; latency and cost on the other. A question that fixes one side tells you which way you may turn.

This is the whole objective in one habit. A 3.3 question always hands you a **constraint** — an SLA, a cost ceiling, an accuracy floor. That constraint **fixes** one currency. The wrong options spend that fixed currency to buy the other one ('add reranking' when the SLA is fixed; 'drop to a smaller model' when the accuracy floor is fixed). The right option moves a dial in the only direction the constraint allows, and can say why in one sentence with a number in it.

Kaveri's six dials, with their one-sentence justifications

Dial	Kaveri setting	The one-sentence justification
Model tier per route	Router: Haiku-class · drafts: Sonnet-class · refunds/agent tier: Opus-class	Spend capability where errors are expensive, not where volume is high
Retrieval depth (top-k)	k=20 after reranking for SOP answers; k=3 for quick lookups	Anthropic's data: 20 beat 5 and 10; but drafts that quote one section need 3
Rerank or not	ON for SOP answers (+~200 ms); OFF inside the 2 s router path	A 67% failure-rate cut is worth 200 ms everywhere the SLA can afford it
Context per call	Cached SOP preamble + top-k chunks; never the whole corpus on the hot path	Tokens are latency AND money; the cache pays only above the tier minimum
Timeouts + retries	Carrier APIs: 3 s timeout, one retry, then degrade to cached ETA	A hung integration must cost a known maximum, not an unknown wait
Parallel fan-out	Three carrier checks concurrently (D1's scouts)	Wall clock = slowest scout; the SLA buys concurrency, not model speed

Notice the shape of every justification: it names the **constraint**, the **dial**, the **direction**, and a **number**. 'A more capable model for better accuracy' has none of those — it is a wish with a price tag.

Three worked examples, in plain words

1 · Tiering by consequence — which model where?

Refund recommendations are 9% of volume and a mistake costs about ₹1,800. Delay drafts are 55% of volume and a mistake costs about ₹4 (an apology template). One mid-tier model serves both. The asymmetry *is* the answer: put the large model on refunds (few calls, expensive errors) and the small model on delay drafts (many calls, cheap errors). The blended cost stays under the ₹6 ceiling because the expensive model only runs on 9% of traffic.

PATH	VOLUME	COST OF AN ERROR	MODEL TIER
Delay drafts	55% of volume	₹4 per mistake	Sonnet-class (mid)
Refund recommendations	9% of volume	₹1,800 per mistake	Opus-class (large)
Routing	100% of volume	cheap to get wrong (re-routed)	Haiku-class (small)

Spend capability where errors are expensive, not where volume is high. Blended cost stays under ₹6.

Consequence, not volume, picks the tier.

2 · A fixed SLA — may I add reranking here?

Router path, 2-second SLA. Current p95 = 1.9 s. Proposal: add reranking (+200 ms).



Verdict: reject on THIS path ($1.9 + 0.2 > 2.0$ at p95). Apply reranking on the SOP-answer path instead, where the SLA has slack and the 67% failure cut is worth 200 ms.

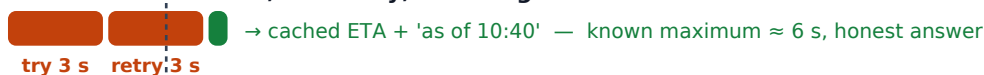
Dials are per path. The same upgrade is right on one route and wrong on another.

3 · A hung dependency — timeouts, retries, honest degradation

AS FOUND — no timeout: a hung carrier API holds the dispatcher up to 40 s



FIXED — 3 s timeout, one retry, then degrade to the cached last-known ETA (labelled stale)



A hung integration must cost a KNOWN maximum, not an unknown wait. (Three retries with backoff can exceed 40 s — no cap.)

Latency engineering, D3-style: timeout + bounded retry + a labelled fallback.

4 · Both options clear the ceiling — now what decides?

Under a ₹6-per-exception ceiling: (A) cache the 8,000-token SOP preamble on the drafting path (hit rate ~92%) → blended cost ₹4.90, accuracy unchanged. (B) shorten the preamble to 900 tokens by summarising → ₹4.60, but draft accuracy drops 3 points. Both fit the ceiling, so **the ceiling stops deciding** and the binding constraint moves to the currency the question did *not* free up: accuracy. Pick A. 'Cheapest wins' optimises past the ceiling into a currency nobody said you could spend.

◎ THE TELL

3.3 stems bait the trade-off-reversed (TR) family: they fix one currency and offer an option that spends it to buy the other — 'add reranking' when the stem fixed a 2-second SLA, 'drop to a smaller model' when it fixed an accuracy floor on refunds. **Underline the fixed currency first; every dial then has a legal direction.**

△ TRAP · JUSTIFYING WITH ADJECTIVES

'A more capable model for better accuracy' is not a justification; it is a wish with a price tag. Exam-grade justification names the constraint, the dial, the direction, and the number: *'the refund path has an accuracy floor and 9% of volume, so it gets the large model; blended cost stays under the ₹6 ceiling.'* If your chosen option cannot be justified in that shape, it is probably the distractor.

△ TRAP · DEFAULTS ARE NOT DECISIONS

'The vector DB defaults to 20' and 'twenty feels comprehensive' are the same non-answer. $\text{top}_k=20$ is right because it was **measured** against 5 and 10 and the token cost fits the path's budget — number, source, constraint.

The five Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q12 · objective 3.3

The router path carries a 2-second end-to-end SLA. An option proposes adding reranking (+200 ms, better SOP answers) inside that path. Current p95 is 1.9 s. Verdict:

A. Add it — accuracy always wins

✓ B. Reject on this path: $1.9 \text{ s} + 0.2 \text{ s}$ breaches the SLA at p95; apply reranking on the non-SLA SOP path instead

C. Add it and raise the SLA

D. Reject reranking everywhere; it is too slow

The tell: 2-second SLA ... current p95 is 1.9 s

Why B is right, in simple words: $1.9 + 0.2 > 2.0$ at p95 — one addition, one breach. Dials are per path: the same upgrade is right on the SOP route and wrong here.

Why the others fall:

A [TR] spends the currency the stem fixed

C [CM] the SLA is the contract, not the variable

D [TR] rejects a 67%-failure-cut dial where latency is slack

Dock Book Q13 · objective 3.3

Refund recommendations (9% of volume) show error costs ~₹1,800 per mistake; delay drafts (55%) cost ~₹4 per mistake. A single mid-tier model currently serves both. The justified 3.3 move:

A. Upgrade everything to the largest model

✓ **B.** Split: large model on refunds, small on delay drafts — spend capability where errors are expensive

C. Downgrade everything to control cost

D. Keep one model; simplicity outranks tuning

The tell: ₹1,800 per mistake ... ₹4 per mistake

Why B is right, in simple words: ₹1,800 vs ₹4 per error — the asymmetry IS the answer. Tiering by consequence is the 3.3 signature move; justify with both numbers.

Why the others fall:

A [OE] pays refund-grade prices for template apologies

C [TR] saves paise on the path that loses thousands

D [SD] 'simplicity' with a number against it is a preference, not a justification

Dock Book Q14 · objective 3.3

Which justification is exam-grade for setting top_k=20 on the SOP path?

A. Twenty feels comprehensive

✓ **B.** Measured: top-20 outperformed top-5 and top-10 on answer quality; the token cost fits the path's budget

C. The vector DB defaults to 20

D. More chunks are always better

The tell: measured ... fits the path's budget

Why B is right, in simple words: Number, source, constraint. 3.3 answers have the shape: metric moved, constraint respected, sentence of why.

Why the others fall:

A [VG] vibes

C [SD] defaults are not decisions

D [TR] past some k, noise displaces signal and cost rises for nothing

Dock Book Q15 · objective 3.3

A carrier ETA API hangs occasionally. Currently: no timeout, calls wait up to 40 s. The dispatcher-facing SLA is 5 s. The 3.3-correct configuration:

A. Retry three times with backoff

✓ **B.** 3 s timeout, one retry, then degrade to the cached last-known ETA with a staleness label

C. Drop the carrier integration

D. Move ETA calls to a bigger model

The tell: no timeout ... SLA is 5 s

Why B is right, in simple words: A hung dependency must cost a known maximum under a 5 s SLA: timeout + bounded retry + honest degradation.

Why the others fall:

A [CM] three retries of a hang can exceed 40 s; no cap on the wait

C [OE-inverse] the data is valuable; the wait is the problem

D [VG] model size does not time out an API

Dock Book Q16 · objective 3.3

Head-to-head under a ₹6/exception blended cost ceiling: (A) cache the 8,000-token SOP preamble on the drafting path (hit rate ~92%); (B) shorten the preamble to 900 tokens by summarising it. Measured: caching lands blended cost at ₹4.90 with unchanged accuracy; the summary lands ₹4.60 but drops draft accuracy 3 points. Pick.

A. B — cheapest wins

✓ **B.** A — both fit the ceiling; only A holds accuracy, so the ceiling stops deciding and accuracy does

C. B — the 3-point drop is negligible

D. Neither; renegotiate the ceiling

The tell: both ... under the ₹6 ceiling

Why B is right, in simple words: Both clear ₹6 — so the binding constraint moves to the next dial, accuracy. When two options both meet the stated constraint, the tie-break is the currency the stem did NOT free up.

Why the others fall:

A [TR] optimises past the ceiling into a currency the stem did not discount

C [CM] the stem gave no licence to spend accuracy

D [SD] nothing needs renegotiating; an option satisfies everything

Carry these into the room

SAY IT IN ONE BREATH

1. Every dial trades accuracy against latency/cost. The stem FIXES one currency — underline it — and every dial then has one legal direction.
2. Justification shape: constraint + dial + direction + number. 'Refunds err at ₹1,800 vs ₹4 → big model there; blended $\leq$ ₹6 holds.'
3. Model tier by consequence, not volume. top-k = 20 (measured) for SOP answers, 3 for quick lookups. Reranking +200 ms for -67% failures — where the SLA can afford it.
4. Timeouts: a hung dependency must cost a known maximum — timeout + one retry + honest, labelled degradation.
5. When both options clear the fixed constraint, the tie-break is the currency the stem did NOT free up.

Next: Part 7 — every number in this part (p95, cost per call, error rates) had to be measured somewhere. The dock cameras are where.

PART 7 OF 8 · OBJECTIVE 3.4

NEW TERRITORY — you cannot fix what you cannot see

The Dock Cameras — Observability at 40,000 a Day

Logs, traces and metrics: three kinds of watching, three different questions — and why the average is where the pain of the unlucky 5% goes to hide

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- What logs, traces and metrics are — as the register, the camera and the gauges — and which question each answers
- What a trace id is and why dropping it turns 40,000 lines into noise without a single error
- Why the mean lied: 310 ms average, 2,900 ms at p95 — and why SLAs are promises about tails
- The cardinality trap (never put an id on a metric label) and symptom-based alerting
- The five-layer stack, the one-minute incident walk, and the six exam questions on this objective

Source: The Loading Dock — Chapter 8 (Objective 3.4), Atlas A6 and Law 4. Written for a reader with very little background: story first, words second, code third.

Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F · Days 5–8 of the 35-day CCAR-P campaign · exam 2 Nov 2026

PART 7 · OBJECTIVE 3.4 · OBSERVABILITY AND MONITORING AT SCALE

The dock cameras — observability at 40,000 a day

A dock that works fine is not a dock you can *prove* works fine. Every number in Parts 2 to 6 — tokens per call, denials by caller, the p95 that decides an SLA — came from the plumbing in this part. Observability is what makes every other argument in this domain numerical instead of rhetorical.

↑ CAMERAS, THE REGISTER, AND THE GAUGES

Three kinds of watching, three different questions. The **register** records what happened, one line per event (**logs** — 'what did truck 4411 do?'). The **camera** that follows one truck gate to bay to gate (**traces** — 'WHERE did truck 4411 lose forty minutes?'). The **gauges** on the wall — trucks per hour, average wait (**metrics** — 'is today normal?'). Lose any one and a class of question becomes unanswerable. At 40,000 trucks, unanswerable means unfixable.

THE REGISTER

structured logs

"What did truck 4411 do?"

one JSON line per event:
{trace, stage, ms, tokens, cost}

THE CAMERA

traces

"WHERE did truck 4411
lose forty minutes?"

one id follows one truck
gate → bay → gate

THE GAUGES

metrics

"Is today normal?"

trucks per hour, p95 wait,
error rate — over time

Logs answer WHAT · traces answer WHERE · metrics answer IS-THIS-NORMAL.

The words, in plain language

Word	In plain words	The dock picture	The exam-grade line
Observability	Being able to answer questions about what the system did, from the outside, after the fact.	Cameras, register and gauges installed before the bad day.	Logs + traces + metrics, instrumented in the emitting code; the precondition for every other D3 number.
Structured logging	Log lines written as data (JSON fields), not sentences.	The register, machine-readable.	One event per line with fields; greppable, aggregatable; free-text logs cannot feed metrics.
Trace	One request's whole journey, stitched together by an id.	Following a single truck gate-to-gate.	A trace id minted at the door and stamped on every event of that request.
Span	One timed step inside a trace.	The stopwatch on one stage: gate, bay, exit.	route / retrieve / draft, each timed alone; answers WHERE time went.
Metric	A number tracked over time, aggregated across many requests.	A gauge on the wall.	Counts, rates, percentiles with a small set of labels; cheap BECAUSE it aggregates.
Cardinality	How many distinct values a metric's label can take.	One gauge per truck = a wall of gauges.	High-cardinality labels (ids!) explode the series count; ids belong in logs and traces, categories in metrics.
Mean vs p95 / p99	The average vs the time 95% (or 99%) of requests beat.	The queue as the unlucky see it.	Percentiles; SLAs are promises about tails. A healthy mean with a sick p95 is the classic hidden failure.

Word	In plain words	The dock picture	The exam-grade line
Dashboard	A screen that renders existing metrics as panels.	The gauges mounted where the foreman can see them.	Renders what exists; creates nothing. Adding panels to a system with no trace ids is decoration.
Alert	An automatic page when a metric crosses a line.	The siren.	Fire on symptoms customers feel (p95, error rate, queue depth); few and dear; everything else is a panel.

The code — the cameras, in 27 lines

The register, the camera and the gauges, small enough to hand-type. Read the story column as one tale: a ticket arrives, is followed through three stages, and at the end of the day the gauges are computed from the register.

```

e7_dock_cameras.py
1  import json, time, uuid, statistics as st
2
3  def log(**fields):                                # the register: one JSON line
4      print(json.dumps(fields, ensure_ascii=False))
5
6  def handle(ticket):
7      trace = str(uuid.uuid4())[:8]                 # the camera follows THIS truck
8      t0 = time.monotonic()
9      for stage, fn in [("route", route_it), ("retrieve", fetch_chunks),
10                       ("draft", draft_reply)]:
11          s0 = time.monotonic()
12          out = fn(ticket)                          # each stage, timed alone
13          log(trace=trace, stage=stage,
14              ms=round((time.monotonic() - s0) * 1000),
15              tokens=getattr(out, "tokens", 0),
16              cost_pc=getattr(out, "paise", 0))      # paise, not vibes
17      log(trace=trace, stage="total",
18          ms=round((time.monotonic() - t0) * 1000))
19
20  def digest(lines):                                # the gauges, from the register
21      by = {}
22      for ln in map(json.loads, lines):
23          by.setdefault(ln["stage"], []).append(ln["ms"])
24      for stage, ms in by.items():
25          ms.sort()
26      p95 = ms[int(0.95 * len(ms)) - 1]
27      print(f"{stage:9s} mean={st.mean(ms):7.0f}ms p95={p95:7.0f}ms")

```

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-4	The story: the dock gets its register. <code>log</code> takes any named fields and prints them as one line of JSON — structured logging : data, not prose. The print stands in for a real log shipper; the <i>shape</i> of the line is the lesson, not the plumbing. <i>Example</i> : instead of 'truck came in around ten and took a while', the register says {"trace": "a1", "stage": "retrieve", "ms": 2850}.
6-8	The camera. A ticket arrives. One trace id is minted (8 random characters) and the clock starts. This id will be stamped on every line this ticket touches — it is what turns 40,000 interleaved lines back into individual journeys. Drop it (tonight's sabotage) and the register becomes noise.

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
9-16	Every stage timed alone — and costed alone. The ticket passes through route → retrieve → draft. Each stage gets its own stopwatch (s0) and its own log line carrying the trace id, the stage name, the milliseconds, the tokens, and — line 16 — the cost in païse . Part 2's bloat audit and Part 6's dials both read from exactly these fields. <i>Example:</i> the camera notes the truck spent 94 ms at the gate, 2,850 ms in the bay, 900 ms leaving.
17-18	The total. One more line, stage 'total', measured from the first clock. Now the register can answer both 'how long did the ticket take?' and 'which part took it?'.
20-27	The gauges, computed FROM the register. Read every line, group the milliseconds by stage, sort each list, take the mean and the value at the 95% position. That is p95: the time 95% of tickets beat — what the unlucky one in twenty actually experienced. The printed block below is the whole objective in four lines: a healthy mean and a sick p95 at the same stage . Averages are where tail pain goes to hide, and SLAs live in the tail.

WHAT PRINTS

```

route    mean=    94ms p95=    180ms
retrieve mean=   310ms p95=  2,900ms    <- the mean lied
draft    mean=   880ms p95=   1,400ms
total    mean=  1,290ms p95=   4,100ms
# 5% of retrievals hit a cold carrier API. The mean hid them; the p95
# is 300 dispatchers' actual Tuesday. SLAs are promises about p95s.
```

One canned day, 10,000 tickets — mean vs p95 per stage (ms)

retrieve: mean 310 ms looks healthy; p95 2,900 ms is 5% of dispatchers' actual Tuesday. The mean lied.

The mean and the p95 of the same stage, drawn to scale.

🔧 TRY IT YOURSELF · PREDICT BEFORE RUN

Run the canned day (10,000 lines, provided). Predict: which stage owns most of the p95, and will any stage's mean look healthy while its p95 is awful? Write the two numbers you expect for 'retrieve' before looking.

Check: retrieve — mean 310 ms, p95 2,900 ms. Nineteen tickets in twenty are fine; the twentieth waits nearly three seconds on a cold carrier API. The average of many quick and a few very slow looks perfectly healthy.

⚡ SABOTAGE LAB · BREAK EXACTLY ONE THING

Mint the trace id **INSIDE** the stage loop (line 7 moved to line 11) so every stage gets a fresh id. The register's line count is identical; predict what question just became unanswerable. Run `digest()` — notice it still works! — then try to reconstruct one slow ticket's journey. **Lesson:** observability gaps don't error; they wait.

40,000 interleaved lines, no trace id

```
{ "stage": "route", "ms": ... } ← whose?
{ "stage": "route", "ms": ... } ← whose?
{ "stage": "retrieve", "ms": ... } ← whose?
{ "stage": "draft", "ms": ... } ← whose?
{ "stage": "retrieve", "ms": ... } ← whose?
{ "stage": "draft", "ms": ... } ← whose?
{ "stage": "route", "ms": ... } ← whose?
{ "stage": "total", "ms": ... } ← whose?
```

*digest() still works — but 'where did ticket a1
lose its time?' is now unanswerable*

The sabotage, drawn. Same lines, same digest, one question lost.

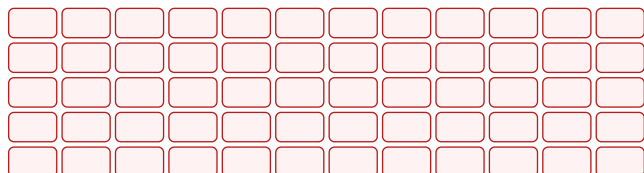
same lines, stamped with a trace id

```
{ "trace": "a1", "stage": "route", "ms": ... }
{ "trace": "b7", "stage": "route", "ms": ... }
{ "trace": "a1", "stage": "retrieve", "ms": ... }
{ "trace": "b7", "stage": "draft", "ms": ... }
{ "trace": "c3", "stage": "retrieve", "ms": ... }
{ "trace": "a1", "stage": "draft", "ms": ... }
{ "trace": "c3", "stage": "route", "ms": ... }
{ "trace": "a1", "stage": "total", "ms": ... }
```

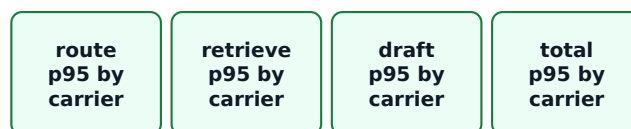
*filter trace=a1 → route 94 · retrieve 2,850 · draft 900
the slow span is found in one query*

Scale: the cardinality trap and symptom-based alerting

The exam's 'at scale' clause has two teeth. **First, cardinality.** Metrics are cheap *because* they aggregate — but only if their label sets stay small. A metric labelled by consignment id is 40,000 new series a day: a wall of gauges nobody can read, and a metrics store that melts. Ids belong in logs and traces; categories (stage, carrier, route) belong in metrics.

latency{consignment_id="KF-2481"} ...

**40,000 new series a DAY — a wall of gauges
nobody can read; the metrics store melts**

latency{stage="retrieve", carrier="BlueDart"}

**a handful of series — categories, not ids.
Ids belong in logs and traces.**

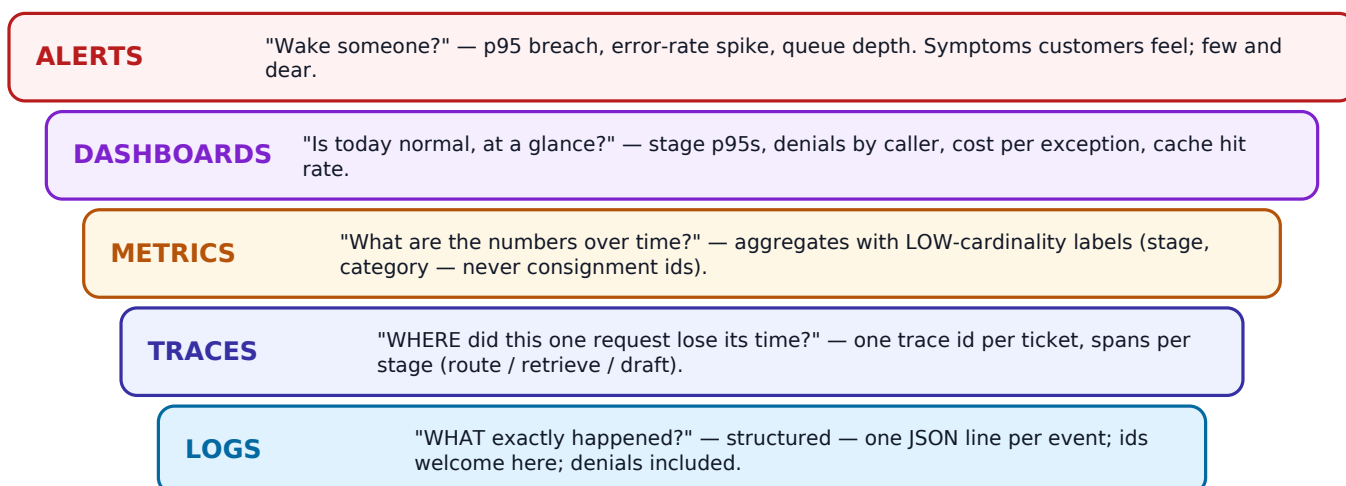
One gauge per truck versus a handful of gauges per category.

Second, alerting. Alert on symptoms customers feel — p95 breach, error rate, queue depth — and keep them few and dear. An alert channel that cries thirty times a day on internal wobbles has trained its readers to scroll; last month a real p95 breach scrolled past unnoticed for six hours. Demote wobbles to dashboard panels.

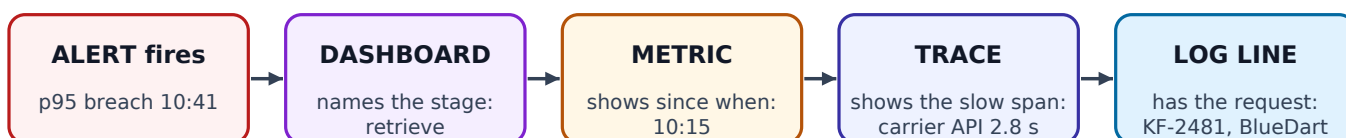
★ LAW 4 · YOU CANNOT FIX WHAT YOU CANNOT SEE

Logs answer WHAT, traces answer WHERE, metrics answer IS-THIS-NORMAL. Instrument stages, stamp a trace id at the door, report p95s not means, keep ids out of metric labels — and **write the log line even when you deny a call**. Every other chapter's numbers came from this one's plumbing.

The five-layer stack and the one-minute incident walk



Bottom feeds top. An incident walks *DOWN* the stack: alert → dashboard → metric → trace → log line. Ids live in the two bottom layers only. Atlas A6. Each layer exists to answer one kind of question.



Five layers, one minute, no guessing. Remove any layer and one step becomes a guess.

The incident walk: alert → dashboard → metric → trace → log line.

Observability serves security too. Part 3's gate writes a log line for every authorization decision, including denials. The 'denials by caller' panel spikes for carrier-portal-v1 at 03:00 while latency and error rates are perfectly normal. That is not noise — it is a probe against the newly narrowed scopes, and the trace ids behind the denials are where to look before anything degrades.

⚠️ TRAP · DASHBOARD THEATER

The sounds-diligent ghost's D3 costume: 'add a dashboard' to a system that emits no trace ids and logs free-text strings. A dashboard renders what exists; it creates nothing. When the gap is instrumentation, the option that adds panels is decoration — the correct option touches the emitting code. (Sibling costume: alerting on 100% of anomalies, which guarantees the one real page gets ignored.)

⚠️ TRAP · 'THE MEAN IS INSIDE BUDGET'

Leadership sees mean 310 ms against a 3 s SLA and declares health. Means and percentiles measure different things by construction: 5% of dispatchers live at 2.9 s in retrieval alone. The SLA is a promise about the tail; report p95s.

The six Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q17 · objective 3.4

Retrieval mean latency is 310 ms; p95 is 2.9 s; the SLA promises 3 s end-to-end. Leadership sees the mean and declares health. The 3.4-correct statement:

- A. The system is healthy; the mean is well inside budget
- ✓ B. 5% of dispatchers live at 2.9 s in retrieval alone — the SLA is at risk at p95, and SLAs are promises about tails
- C. Means and percentiles measure the same thing at different sample sizes
- D. Raise the SLA to match the mean

The tell: mean ... 310 ms; p95 is 2.9 s

Why B is right, in simple words: Mean 310 / p95 2,900 — the tail is where the promise breaks. Report p95s.

Why the others fall:

- A [SF] the mean is where tail pain hides
- C [VG] they measure different things by construction
- D [CM] the SLA is the contract

Dock Book Q18 · objective 3.4

40,000 interleaved log lines per day. Which single missing field makes the question 'where did ticket T lose its time?' unanswerable?

- A. The model version
- ✓ B. A per-request trace id stamped on every stage's line
- C. The caller's scope list
- D. The token count

The tell: where did ticket T lose its time

Why B is right, in simple words: Reconstructing one journey from interleaved lines requires the id that threads them. Logs answer WHAT; only a trace id turns them into WHERE.

Why the others fall:

- A [CM] useful, answers other questions
- C [CM] Part 3's question, not this one
- D [CM] cost questions, not journey questions

Dock Book Q19 · objective 3.4

A proposal: emit a latency metric labelled by consignment id 'for maximum drill-down.' The 3.4 verdict:

- A. Approve; more labels, more insight
- ✓ B. Reject: 40,000 new series/day — the cardinality trap; ids belong in logs and traces, categories in metrics
- C. Approve but sample 10% of ids
- D. Reject metrics entirely; logs suffice

The tell: labelled by consignment id

Why B is right, in simple words: An id as a metric label is a gauge per truck, forever. Cardinality discipline: id → log/trace; category → metric.

Why the others fall:

- A [OE/VG] melts the metrics store
- C [CM] 4,000 series/day is still the trap, slower
- D [TR] throws away trends to dodge a labelling mistake

Dock Book Q20 · objective 3.4

The alert channel fires ~30 times a day on internal wobbles; last month a real p95 breach scrolled past unnoticed for 6 hours. The 3.4 fix:

A. Add more alerts so nothing is missed

✓ **B.** Alert only on customer-felt symptoms (p95, error rate, queue depth); demote wobbles to dashboards

C. Route alerts to more people

D. Silence the channel during business hours

The tell: ~30 times a day ... scrolled past unnoticed

Why B is right, in simple words: A channel that cries thirty times a day has trained its readers to scroll. Symptom-based alerting: few, dear, actionable.

Why the others fall:

A [OE] more noise is not more signal

C [SD] wider distribution of noise is still noise

D [SF] institutionalised blindness

Dock Book Q21 · objective 3.4

Which pairing correctly matches question to layer?

A. 'What exactly happened on call X?' → metrics; 'Is today normal?' → logs

✓ **B.** 'What exactly happened?' → structured logs; 'Where did the time go?' → traces; 'Is today normal?' → metrics

C. All three questions → the dashboard

D. 'Where did the time go?' → alerts

The tell: question to layer

Why B is right, in simple words: Three questions, three layers — the A6 stack verbatim. WHAT / WHERE / NORMAL → logs / traces / metrics.

Why the others fall:

A [VG] reversed

C [SD] dashboards render layers; they are not the layers

D [VG] alerts are triggered symptoms, not diagnosis

Dock Book Q22 · objective 3.4

The denials-by-caller dashboard (built in Part 3) spikes for carrier-portal-v1 at 03:00. Latency and error rates are normal. The 3.4/3.2-correct reading:

A. Noise; nothing customer-facing degraded

✓ **B.** A possible probe against narrowed scopes — investigate the trace ids behind the denials before anything degrades

C. The scope narrowing was too strict; widen it

D. Restart the portal integration

The tell: denials ... spike at 03:00 ... latency normal

Why B is right, in simple words: Denials spiking with healthy latency is a security signal, not a performance one. The denial panel exists for exactly this 03:00.

Why the others fall:

A [SF] 'nothing degraded' is the attacker's favourite dashboard

C [OP] widening scopes because someone is knocking is opening the door

D [SD] restarts cure state, not intent

Carry these into the room

SAY IT IN ONE BREATH

1. Logs = WHAT (one JSON line per event, ids welcome, denials included). Traces = WHERE (one trace id per request, spans per stage). Metrics = IS-THIS-NORMAL (aggregates, LOW-cardinality labels).
2. Mean 310 / p95 2,900 = the mean lied. SLAs are promises about p95s, not means.
3. Ids NEVER label metrics (40,000 series a day). Categories in metrics; ids in logs and traces.
4. Alert on symptoms customers feel — few and dear. Everything else is a dashboard panel.
5. The incident walk: alert → dashboard → metric → trace → log line. A dashboard renders what exists; the fix for missing instrumentation is in the emitting code.

Next: Part 8 — the last dial of all: what rides in every prompt versus what is fetched on demand, priced.

PART 8 OF 8 · OBJECTIVE 3.8

SOFT — WHERE ALL OF DOMAIN 3'S ECONOMICS RECONVERGE

The Catalogue at the Gate — Progressive Discovery vs the Monolith

Hand the driver the catalogue, not the warehouse — unless the warehouse is smaller than the catalogue. What rides in every prompt, what is fetched on demand, and the one exception you learn by heart

WHAT YOU WILL BE ABLE TO SAY AFTER READING THIS PART

- What 'monolithic context' and 'progressive discovery' mean, as two ways to load a truck
- Two bills for the same question, line by line: 362,460 tokens vs 4,625 tokens (+ one round-trip)
- The exception you must know cold: small + stable + always-used + cacheable → the monolith WITH caching wins
- What prompt caching and server-side context editing do, and what each one costs
- The trap on both sides — 'progressive everything' and 'reading only the token column' — and the five exam questions

Source: The Loading Dock — Chapter 9 (Objective 3.8), Atlas A7 and Law 5. Written for a reader with very little background: story first, words second, code third.

Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F · Days 5-8 of the 35-day CCAR-P campaign · exam 2 Nov 2026

PART 8 · OBJECTIVE 3.8 · PROGRESSIVE DISCOVERY VERSUS MONOLITHIC CONTEXT

The catalogue at the gate — progressive discovery vs the monolith

This objective is where all of Domain 3's economics reconverge. Part 2 priced tool schemas; Part 4 priced retrieval. This part prices the strategy choice itself: what rides in every prompt, and what is fetched only when needed.

↑ HAND THE DRIVER THE CATALOGUE, NOT THE WAREHOUSE

A driver arrives for one pallet. **Warehouse A** loads his truck with everything he MIGHT need — he drives away slow, overweight, and mostly wrong. **Warehouse B** hands him a one-page catalogue at the gate; he asks for bay 12, gets bay 12. But note the fine print: for a tiny warehouse with three shelves that every driver uses every time, the catalogue is pure overhead — just load the three shelves. **The map first, the territory on demand — unless the territory is smaller than the map.**

WAREHOUSE A — load everything he MIGHT need

9 full tool manuals

all 400 SOPs

the question

Slow, overweight, mostly wrong: 362k tokens for a question that needs one tool and one section.

WAREHOUSE B — the catalogue at the gate

one-page catalogue (9 lines)

the one manual he asked for

top-20 boxes

the question

4.6k tokens — plus one trip back to the gate to ask for bay 12. Light, but never free.

Two ways to load the same truck for the same one-pallet job.

The words, in plain language

Word	In plain words	The dock picture	The exam-grade line
Monolithic context	Everything stuffed into every request: all tool schemas, all documents, all references.	Loading the whole warehouse on the truck.	All schemas + corpus every call; correct ONLY when small + stable + always-used + cached (corpus $\leq$ 200k tokens).
Progressive discovery / disclosure	Send a light index first; fetch details only when the task needs them.	The catalogue at the gate.	Deferred loading of schemas/content; trades an extra round-trip for a small standing context.
Catalogue	One line per tool — its name and when to use it — instead of its full schema.	The one-page list at the gate.	~25 tokens per tool vs ~260 for a full schema; the full schema is fetched when the tool is chosen (tools/list → tools/call).
Round-trip	One extra exchange with the model to fetch what it chose.	The trip back to the gate to ask for bay 12.	The latency price of progressive discovery; the one cost the catalogue can never refund.
Prompt caching	Re-using a large fixed prefix so it is read at ~10% of the price.	The laminated preamble that every desk already has.	Cache reads ~90% off after the first write; minimum cacheable prefix 1,024–4,096 tokens by tier (Haiku 4.5 = 4,096).

Word	In plain words	The dock picture	The exam-grade line
Context editing	The API trims old tool results server-side before the model reads them.	Sweeping the desk between jobs.	clear_tool_uses_20250919 (trigger ~100k input tokens, keep 3 recent pairs, exclude_tools) + clear_thinking_20251015; clearing can invalidate cache — pair with clear_at_least and the memory tool.
Room to think	The part of the window left for the model's own reasoning and answer.	The empty floor space on the truck.	The monolith's hidden cost: a thin slice of attention, and retrieval quality degrading as the window fills with irrelevant text.

The code — two bills for the same task

One dispatcher question that needs one tool and one SOP section. Price it three ways. The gap is the point — get it in your hands.

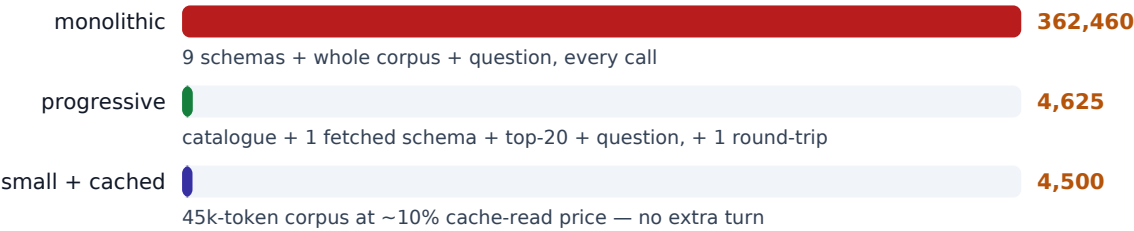
```
e8_two_bills.py
1  # Task: answer one dispatcher question that needs 1 tool + 1 SOP section
2  MONO = {
3      "tool_schemas": 9 * 260,          # every forklift's manual, every call
4      "sop_corpus": 400 * 900,          # the whole warehouse, every call
5      "question": 120,
6  }
7  PROG = {
8      "catalogue": 9 * 25,              # one line per tool: name + when-to-use
9      "fetch_tool": 260,                # full schema, fetched on demand
10     "retrieved": 20 * 180,             # top-20 chunks, not 400 documents
11     "question": 120,
12     "extra_turn": 1,                  # the fetch costs a round-trip
13 }
14 for name, bill in [("monolithic", MONO), ("progressive", PROG)]:
15     toks = sum(v for k, v in bill.items() if k != "extra_turn")
16     print(f"{name:12s}: {toks:8,d} tokens/call"
17           f"' + 1 round-trip' if bill.get('extra_turn') else ''")
18
19 # When the monolith WINS: corpus under ~200k tokens, stable, always-used
20 CACHED_SMALL = {"corpus": 45_000, "cache_hit_price": 0.1} # 90% off after write
21 print(f"small+cached: {int(45_000 * 0.1):8,d} token-equivalents/call, no extra turn")
```

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
1-6	The story: one question, three ways to pay for it. The monolith's bill: nine full schemas (Part 2's ~260 tokens each) plus the entire SOP corpus (400 docs × ~900 tokens), plus the 120-token question — on EVERY call, for a question that needs one tool and one section. 362,460 tokens: most models' entire window, spent on insurance. <i>Example:</i> Warehouse A loads every manual and every shelf for a one-pallet job.
7-13	The catalogue. One line per tool ('name + when to use', ~25 tokens) — the full 260-token schema is fetched only when the model chooses that tool. This is exactly how deferred-tool systems and MCP's tools/list → tools/call shape work. And Part 5's top-20 retrieved chunks (20 × 180) replace the 400-document dump. Price: one extra round-trip when a fetch happens (line 12). Progressive discovery trades latency for context — a trade, not a free lunch, which is why 3.8 says 'evaluate', not 'always'.
14-17	Both bills printed. Sum everything except the round-trip flag. 362,460 vs 4,625 — 78× is the kind of number that ends meetings. But the exam wants the NEXT line too.

Lines	LINE BY LINE — ONE STORY, SIMPLE WORDS
19-21	The monolith's honest case. A corpus under ~200k tokens (the skip-RAG threshold from Part 4), stable and always relevant, parked behind prompt caching : cache reads are ~90% off, so 45,000 tokens cost about 4,500 token-equivalents — near-progressive cost, and zero extra turns . <i>Example:</i> the tiny warehouse with three shelves every driver uses: just load the shelves; the catalogue would only add a trip. Small, stable, always-used → stuff and cache. Large, changing, occasionally-used → catalogue and fetch.

WHAT PRINTS
monolithic : 362,460 tokens/call
progressive : 4,625 tokens/call + 1 round-trip
small+cached: 4,500 token-equivalents/call, no extra turn
78x between the first two. The third line is the monolith done RIGHT:
small stable corpus + prompt caching ~= progressive costs, zero latency tax.

Three bills for the same dispatcher question (tokens per call, to scale)

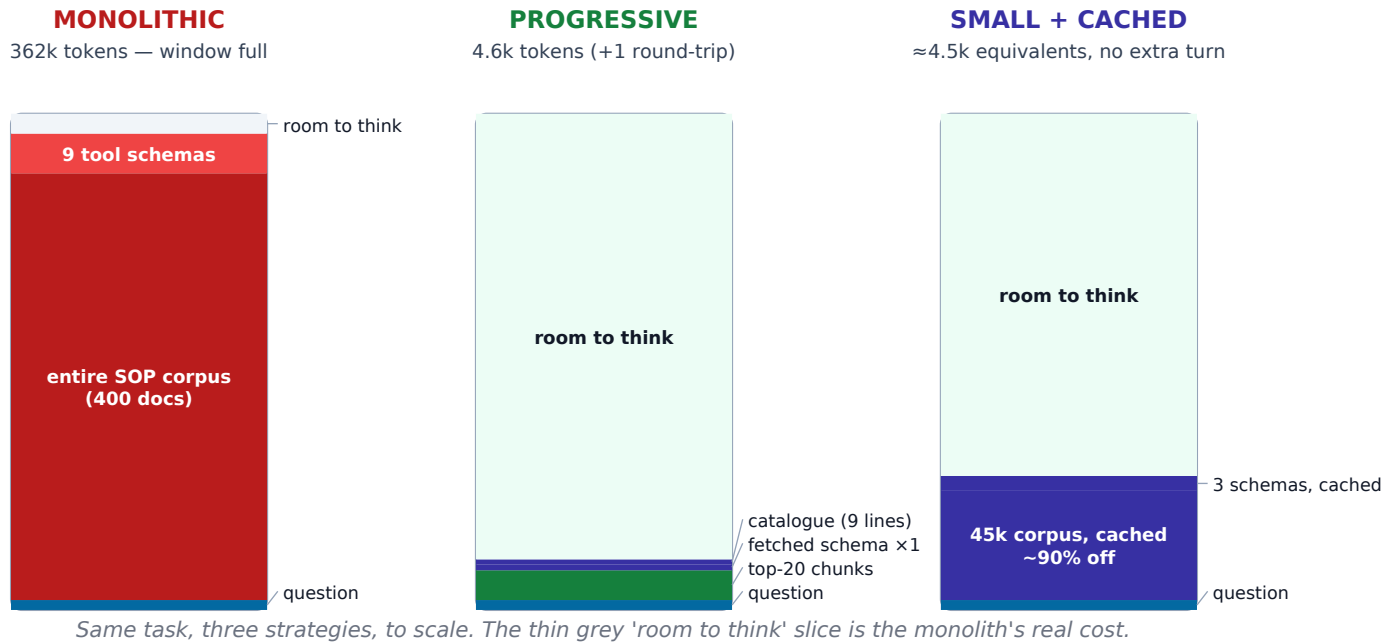


78x between the first two. The third line is the monolith done RIGHT — near-progressive cost, zero latency tax.
Three bills, to scale.

TRY IT YOURSELF · PREDICT BEFORE RUN
Sum both bills in the notebook first — the gap is the point. Then predict: at what corpus size and stability does the third line beat the second, and which two D3 facts (one from Part 4's DRIFT, one from caching) decide it?
Check: the ~200k-token skip-RAG threshold, and the ~90% cache-read discount above the tier's minimum cacheable prefix. Together: a small, stable, always-used corpus costs about the same as progressive fetching — without the round-trip.

The window, drawn — Atlas A7

The bar is the model's context window; what you fill it with is what you pay for — in tokens, and in the model's attention.



⚠ TRAP · READING ONLY THE TOKEN COLUMN

The monolith's real cost is not only the bill — it is the thin 'room to think' slice, and retrieval quality degrading as the window fills with mostly-irrelevant text. The progressive bar's real cost is the round-trip and the risk of under-fetching. 3.8 options that mention only one side's cost are advertising, not analysis.

The decision, as a grid — and the exception you learn by heart

	LARGE ($\geq 200k$ tokens)	SMALL ($\leq 200k$ tokens)
CHANGES OFTEN / USED SOMETIMES	LARGE · CHANGING · OCCASIONALLY USED CATALOGUE + FETCH 400 SOPs, 30 aggregated tools: send the map, fetch on choice	SMALL · CHANGING Fetch, or re-cache per version a small pack that changes daily — caching keeps missing
STABLE / ALWAYS USED	LARGE · STABLE · ALWAYS USED Retrieval (top-20) + cached preamble 1.4M tokens cannot ride in any window	SMALL · STABLE · ALWAYS USED MONOLITH + PROMPT CACHING 45k policy pack, quarterly changes, every call: stuff it and cache it

The bottom-right cell is the exception the exam tests: three words — small, stable, always-used — flip the default.

Size on one axis, stability/usage on the other. The bottom-right cell is the flip.

★ LAW 5 · SEND THE MAP FIRST, THE TERRITORY ON DEMAND

Light indexes up front; details fetched when chosen; retrieval instead of corpus-stuffing; context editing to shed exhaust in long runs. **Exception, learned by heart:** small + stable + always-used + cacheable → monolith WITH caching wins, because the fetch's round-trip is the one cost the catalogue can never refund.

⚠️ TRAP · PROGRESSIVE EVERYTHING

3.8's inverted distractor: catalogue-and-fetch applied to three tools and a 40k-token policy pack that every single call uses. Now every request pays a discovery round-trip to avoid a context cost that caching had already reduced ~90%. The tell is **small, stable, always-used** — when you read those three properties together, the monolith-plus-cache is the correct answer and the fashionable one is not.

A worked example. KAVI's agent tier sees 30 tools across aggregated MCP servers, but any single request uses at most 3. Full schemas: $30 \times 260 = 7,800$ standing tokens per call. A one-line catalogue: $30 \times 25 = 750$, plus up to 3 fetched schemas (780) = about **1,500**. Large-ish, changing, occasionally used → catalogue. Now the opposite: a 45k-token policy pack, stable for quarters, consulted by essentially every call. Fetching it progressively 'as best practice' pays a round-trip on every request to avoid a bill caching already cut ~90%. Small, stable, always-used → stuff it and cache it.

Long runs — context editing, server-side

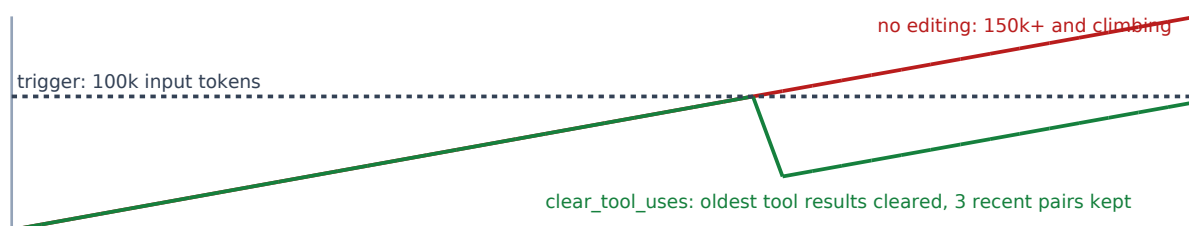
An agent-tier investigation runs 40 turns; old tool results pile up until input passes 150k tokens and costs climb. The old advice was 'summarise old turns yourself, client-side'. The current API does it **server-side**:

⇌ DRIFT · THE DOCS MOVED

What older teaching says: When the window fills, summarise old turns yourself, client-side.

What the current docs say (checked Sep 2026): Context editing runs server-side (beta header context-management-2025-06-27): `clear_tool_uses_20250919` clears the oldest tool results on a trigger (default 100k input tokens; keep the default 3 recent pairs; `exclude_tools` protects named tools), and `clear_thinking_20251015` manages thinking blocks. Clearing can invalidate cached prefixes — `clear_at_least` exists to make the invalidation worth it — and the `memory_20250818` tool lets the model save essentials to files before they are cleared. Your client keeps the full history; the trim happens before Claude reads it.

A 40-turn agent investigation, input tokens over time



*Each clear invalidates the cached prefix from that point — one cache re-write, then cheaper turns.
`clear_at_least` makes the trade worth it; the memory tool saves essentials first.
 Input tokens across a 40-turn run, with and without server-side clearing.*

The honest cost accounting (Dock Book Q44): clearing 60k tokens of old tool results is not free — the cleared prefix invalidates the cache at that point, so you pay one cache re-write, then cheaper turns. That is why `clear_at_least` exists: clear enough in one go that the re-write is worth it. Every 3.8 mechanism has a bill; naming it is what 'evaluate' means.

The five Dock Book questions on this objective, explained

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

Dock Book Q41 · objective 3.8

30 tools across aggregated MCP servers; any single request uses at most 3. Schemas cost ~260 tokens each. The 3.8-correct context strategy:

- A. All 30 schemas on every call; completeness first
- ✓ B. A one-line catalogue of all 30; full schemas fetched on demand — ~7.8k standing tokens become ~1.5k
- C. Drop to 3 tools permanently
- D. Cache the 30 schemas and stop thinking about it

The tell: any single request uses at most 3

Why B is right, in simple words: Any request uses at most 3 — the catalogue's exact use case. Send the map first; fetch on choice.

Why the others fall:

- A [OE] 7.8k tokens of standing insurance
- C [CM] the other 27 are used, just not together
- D [CM] caching discounts the bill; attention and window costs remain

Dock Book Q42 · objective 3.8

A 45k-token policy pack is stable for quarters and consulted by essentially every call. Progressive fetching is proposed 'as best practice.' Verdict:

- A. Adopt; progressive is always leaner
- ✓ B. Reject: small + stable + always-used → monolith WITH prompt caching; the fetch round-trip is pure loss here
- C. Adopt half: fetch the pack every second call
- D. Reject caching and fetching; paste summaries

The tell: 45k-token ... stable for quarters ... essentially every call

Why B is right, in simple words: Small, stable, always-used — the three words that flip the default. The exception is on the Memory Map because the exam tests the flip, not the fashion.

Why the others fall:

- A [TR] pays a round-trip to avoid a bill caching already cut ~90%
- C [VG] incoherent hybrid
- D [SF] summaries silently drop the clause the dispute needs

Dock Book Q43 · objective 3.8

An agent-tier investigation runs 40 turns; old tool results push input past 150k tokens and costs climb. The current-API answer:

- A. Restart the conversation every 10 turns
- ✓ B. Server-side context editing: clear_tool_uses on a token trigger, keep recent pairs, exclude critical tools; pair with the memory tool so essentials survive clearing
- C. Summarise manually client-side each turn
- D. Raise max_tokens

The tell: old tool results push input past 150k

Why B is right, in simple words: Old tool results are the named clearable class; the API now trims them server-side. clear_tool_uses_20250919 + the memory tool: the long-run pattern, by name.

Why the others fall:

- A [SF] restarts amputate the investigation's memory
- C [CM] workable but ignores the platform mechanism the stem points at
- D [VG] output budget; unrelated to input bloat

Dock Book Q44 · objective 3.8

Context editing clears 60k tokens of old tool results mid-run. What is the honest cost accounting?

A. Free; cleared tokens were waste

✓ **B.** The cleared prefix invalidates cache at that point — one cache re-write, then cheaper turns; `clear_at_least` exists to make the trade worthwhile

C. Clearing corrupts the model's memory of its own reasoning

D. It doubles latency permanently

The tell: honest cost accounting

Why B is right, in simple words: Clearing invalidates cached prefixes — a priced trade, not a free lunch. Every 3.8 mechanism has a bill.

Why the others fall:

A [VG] the cache interaction is the whole nuance

C [VG] the client keeps history; thinking-block policy is separate and configurable

D [VG] one re-write, not a permanent tax

Dock Book Q45 · objective 3.8

Head-to-head, D1 crossover. A nightly batch agent (400 cases) needs the SOP corpus. (B) monolithic corpus in every case's context; (C) per-case hybrid retrieval, top-20. Numbers: corpus 1.4M tokens (over any window); retrieval adds 180 ms/case; the batch window is 4 hours. Pick.

A. B — batches prefer simplicity

✓ **B.** C — 1.4M tokens cannot ride in any context; retrieval's $180 \text{ ms} \times 400 = 72 \text{ s}$ against a 4-hour window

C. B — with the corpus split across turns

D. Neither; skip the SOPs at night

The tell: 1.4M tokens (over any window)

Why B is right, in simple words: First check feasibility against the window (1.4M tokens ends option B before taste begins), then price the survivor: 72 seconds inside 4 hours. Two numbers, done.

Why the others fall:

A [CM] physically impossible before it is stylistically debatable

C [SF] corpus-across-turns is monolith cosplay with worse recall

D [CM] the cases need the SOPs; that is the task

Carry these into the room — and the five laws of Domain 3

📖 SAY IT IN ONE BREATH

1. Law 5: the map first, the territory on demand — EXCEPT small + stable + always-used + cacheable ($\leq 200\text{k}$ tokens) → monolith + caching wins.
2. Two bills: monolithic 362,460 vs progressive 4,625 (+1 round-trip) vs small+cached $\approx 4,500$ with no extra turn. 78×, then the next line.
3. Progressive discovery trades latency (a round-trip) for context; 3.8 says 'evaluate', not 'always'. Options that price only one side are advertising.
4. Caching: reads $\sim 90\%$ off; minimum cacheable prefix 1,024–4,096 by tier (Haiku 4.5 = 4,096).
5. Context editing (server-side): `clear_tool_uses_20250919` — trigger $\sim 100\text{k}$, keep 3, `exclude_tools`, `clear_at_least`; `clear_thinking_20251015`; clearing can invalidate cache; memory tool saves essentials first.

#	The law	Where it lives
1	Every tool you add is a door; know who holds the key.	Part 2 (bloat) + Part 3 (auth)
2	Chunk along meaning, not length.	Part 4
3	Retrieval follows the shape of the data, not the fashion.	Part 5
4	You cannot fix what you cannot see.	Part 7
5	Send the map first, the territory on demand.	Part 8

The whole Dock in one sentence: 'It worked in the demo' is the distance between one user, ten documents and one taped-over API key — and forty dispatchers, four hundred documents, three carriers and nobody watching. Every Domain 3 question is some version of that distance.